TOKAT GAZİOSMANPAŞA ÜNİVERSİTESİ
Erbaa Meslek Yüksekokulu

TOKAT GAZİOSMANPAŞA ÜNİVERSİTESİ

Erbaa Meslek Yüksekokulu

Bilgisayar Teknolojileri Bölümü

Bilgisayar Programcılığı Programı

Mesleki Çözümleme II Dersi Yarıyıl Sonu Proje Raporu

ShineBeauty

241909006
241909040
Sefa TEMİZ
Ufuk COŞKUN

Dr. Ögr. Üyesi Mehmet Fatih KARACA

01/06/2026

İÇİNDEKİLER

İÇİNDEKİLER....2
PROJE BİLGİLERİ....3
ÖZET....4
1. GİRİŞ VE PROJENİN AMACI....5
2. KULLANILAN TEKNOLOJİLER VE MİMARİ ÇÖZÜMLER....8
3. VERİ TABANI TASARIMI....12
4. SİSTEM MODÜLLERİ VE KOD MİMARİSİ....18
5. GÜVENLİK VE YETKİLENDİRME....34
6. SONUÇ VE GELECEK GELİŞTİRMELER....39
KAYNAKÇA....43

PROJE BİLGİLERİ

Projenin Adı : ShineBeauty

Projenin Amacı : Bu projenin temel amacı; bir güzellik merkezinin uçtan uca tüm operasyonel, finansal ve idari süreçlerini dijitalleştiren, merkezi, entegre ve tam otomatize edilmiş bir Yönetim Bilgi Sistemi geliştirmektir. Yazılım, işletmenin verimliliğini maksimize etmeyi, operasyonel maliyetleri düşürmeyi ve insan faktöründen kaynaklanan hata payını algoritmik kısıtlamalarla sıfıra indirmeyi hedeflemektedir.

Projenin Hedef Kitlesi : Projenin hedef kitlesi; günlük operasyon yoğunluğu yüksek olan güzellik merkezleri, estetik ve bakım klinikleri, kuaför salonları ve spa merkezleridir. Aynı zamanda randevu, stok, personel ve finans takibini tek bir merkezi sistem üzerinden yürüterek veri kayıplarını önlemek ve müşteri memnuniyetini artırmak isteyen tüm orta ve büyük ölçekli hizmet işletmeleri bu projenin doğal kullanıcıları arasında yer almaktadır. Sistemin rol bazlı yapısı sayesinde, işletme sahipleri ve yöneticilerin yanı sıra, sahada aktif olarak hizmet veren personeller ve ön büro çalışanları da projenin doğrudan hedef kitlesini oluşturmaktadır.

Projenin Özeti : ShineBeauty projesi, hizmet sektöründeki işletmelerin karmaşık operasyonel ve finansal iş akışlarını dijitalleştiren, Node.js ve MSSQL mimarisi üzerine inşa edilmiş entegre bir Yönetim Bilgi Sistemidir. Randevu optimizasyonu, stok takibi, paket seans yönetimi ve kasa hareketlerini birbirine bağlı otomatize modüllerle yönetir. Sistem, rol bazlı güvenlik katmanları ve atomik veritabanı işlemleri sayesinde veri bütünlüğünü garanti altına alırken, insan faktöründen doğabilecek hataları proaktif algoritmalarla engelleyerek işletme verimliliğini en üst düzeye çıkarmaktadır.

ÖZET

ShineBeauty projesi, modern hizmet sektörünün gereksinim duyduğu yüksek işlem hacmini ve karmaşık veri ilişkilerini yönetmek üzere tasarlanmış kapsamlı bir Yönetim Bilgi Sistemidir. Geleneksel ve manuel olarak yürütülen randevu, finans, stok ve personel süreçlerini tek bir çatı altında toplayan bu sistem, arka planda çalışan güçlü algoritmaları sayesinde insan hatasını en aza indiren otomatize bir yapı sunmaktadır.

Sistem mimarisi, yüksek performans ve asenkron işlem kabiliyeti sunan Node.js ve Express altyapısı üzerine inşa edilmiş olup, veri bütünlüğü ve güvenliği ilişkisel veritabanı yönetim sistemi olan MSSQL ile güvence altına alınmıştır. Proje kapsamında geliştirilen randevu modülü, yalnızca tarih ve saat kaydı tutmakla kalmayıp; seçilen hizmetin gereksinimlerine göre uygun oda filtrelemesi yapan ve personel çakışmalarını engelleyen proaktif bir mekanizmaya sahiptir. Bu sayede işletme içindeki kaynak yönetimi kusursuz bir işleyişe kavuşturulmuştur.

Finans, paket ve stok modülleri ise birbirleriyle tam entegre çalışacak şekilde tasarlanmıştır. Satın alınan ürünlerin stok maliyetlerinin anlık olarak işletme giderlerine yansıtılması, satılan hizmet paketlerinin otomatik olarak ciroya eklenmesi ve her randevu bitiminde ilgili seansın müşterinin paketinden düşülmesi gibi operasyonlar, veri tabanı seviyesinde uygulanan atomik işlemlerle garanti altına alınmıştır. Güvenlik katmanında ise oturum tabanlı rol bazlı erişim kontrolü kullanılarak, yönetici ve personel yetkileri kesin sınırlarla ayrılmış, yetkisiz veri erişimlerinin tamamen önüne geçilmiştir.

Sonuç olarak bu proje, kapsamlı kod mimarisi ve birbirine sıkı sıkıya bağlı modüler yapısıyla, bir güzellik merkezinin tüm yaşamsal fonksiyonlarını dijital ortama taşıyan, veri tutarlılığını esas alan ve işletme verimliliğini somut olarak artıran profesyonel bir yazılım çözümüdür. Sistem, sunduğu şeffaf yönetim kokpiti ile işletme sahiplerine veriye dayalı stratejik kararlar alma imkanı sunmaktadır.

1. GİRİŞ VE PROJENİN AMACI

ShineBeauty projesi, güzellik merkezi, estetik salonu, kuaför, bakım kliniği ve benzeri hizmet işletmelerinde günlük olarak yürütülen randevu, müşteri, personel, oda, hizmet, ürün, stok, paket seans ve finans süreçlerini tek bir sistem altında toplamak amacıyla geliştirilmiş web tabanlı bir yönetim bilgi sistemidir. Bu proje, işletme içinde manuel olarak takip edilen işlemleri dijital ortama taşıyarak hem veri kayıplarını önlemeyi hem de işletmenin daha düzenli, ölçülebilir ve kontrol edilebilir şekilde yönetilmesini sağlamayı hedeflemektedir.

Geleneksel güzellik merkezi yönetiminde müşteri kayıtları, randevu saatleri, personel uygunluğu, oda kullanımı, hizmet süresi, ürün satışı, stok takibi ve kasa hareketleri çoğu zaman ayrı ayrı defterlerde, mesajlaşma uygulamalarında veya birbirinden kopuk dosyalarda tutulabilmektedir. Bu durum, aynı saat aralığına birden fazla randevu verilmesi, personelin uygun olmadığı bir zamana işlem atanması, odaların çakışması, paketi olmayan müşteriye randevu oluşturulması, stokta olmayan ürünün satılması veya yapılan satışın kasaya doğru yansıtılmaması gibi hatalara sebep olabilmektedir. ShineBeauty projesi, bu sorunları azaltmak için işlemleri merkezi bir veritabanı üzerinde birleştiren ve arka planda çeşitli kontroller yapan bir yapı üzerine kurulmuştur.

Projenin temel amacı, bir güzellik merkezinin operasyonel süreçlerini uçtan uca dijitalleştirmek ve işletme sahibine daha güvenilir bir yönetim paneli sunmaktır. Sistem yalnızca basit bir kayıt tutma uygulaması olarak düşünülmemiştir. Randevu oluşturma sırasında hizmet süresi, seçilen personel, oda uygunluğu, müşteri paketi, tarih ve saat bilgisi birlikte değerlendirilmekte; böylece aynı kaynakların aynı zaman aralığında birden fazla işlem için kullanılmasının önüne geçilmektedir. Bu yönüyle proje, işletme içinde oluşabilecek zaman ve kaynak çakışmalarını önlemeye çalışan kontrollü bir randevu yönetimi mantığına sahiptir.

Projenin bir diğer amacı, müşteri ilişkilerini daha düzenli hale getirmektir. Sistemde müşterilerin ad, soyad, telefon, e-posta, cinsiyet, doğum tarihi, müşteri tipi, T.C. kimlik numarası,

notlar ve son randevu bilgileri tutulabilmektedir. Bu bilgiler sayesinde işletme, müşterinin geçmiş işlemlerini, randevu durumlarını, paket kullanımını ve iletişim bilgilerini tek yerden takip edebilmektedir. Ayrıca uzun süredir gelmeyen müşterilerin listelenebilmesi, işletmenin müşteri devamlılığını analiz etmesine yardımcı olmaktadır.

ShineBeauty projesinde yalnızca yönetici tarafı değil, müşteri tarafı da düşünülmüştür. Kullanıcılar, giriş yapmadan web sitesi üzerinden hizmetleri ve ürünleri görebilmekte, uygun hizmet seçeneğini belirleyerek online randevu talebi oluşturabilmektedir. Ancak müşteri tarafından oluşturulan randevular doğrudan aktif hale getirilmemektedir. Bu randevular sistemde "Bekliyor" durumunda tutulmakta ve yetkili kullanıcı tarafından kontrol edildikten sonra onaylanmaktadır. Bu yapı, işletmenin kontrolünü kaybetmeden dışarıdan randevu talebi alabilmesini sağlamaktadır.

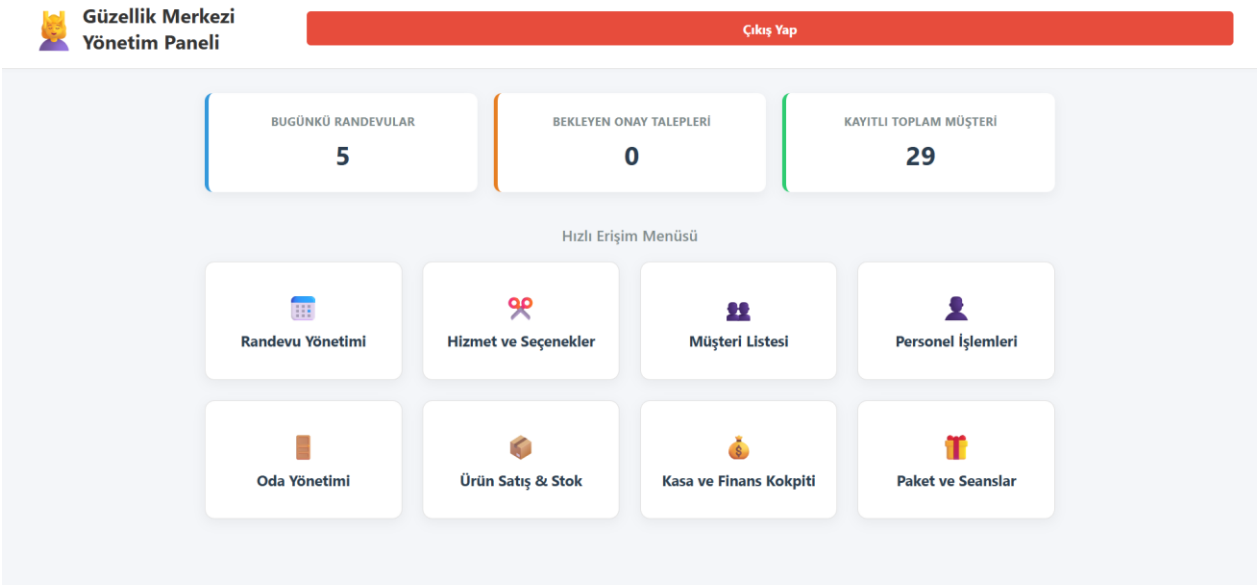
Projenin finansal amacı ise gelir ve gider takibini daha şeffaf hale getirmektir. Ürün satışları, paket satışları, gider kayıtları ve kasa hareketleri ayrı tablolar üzerinden tutulmakta, ancak finans ekranında bir bütün olarak değerlendirilmektedir. Ürün satıldığında stok düşmekte ve satış tutarı gelir olarak kasaya işlenmektedir. Paket satıldığında paket kaydı oluşturulmakta ve aynı işlem kasa hareketlerine gelir olarak aktarılmaktadır. Yeni ürün stoğu eklendiğinde stok maliyeti gider olarak kaydedilebilmektedir. Böylece sistem yalnızca operasyonel süreci değil, aynı zamanda işletmenin parasal hareketlerini de izlenebilir hale getirmektedir.

Bu proje, yönetici ve personel rollerini ayırarak işletme içinde yetki kontrolü sağlamayı da amaçlamaktadır. Admin rolündeki kullanıcılar personel, hizmet, oda, ürün ve sistemsel kayıtlar üzerinde daha geniş yetkilere sahipken; personel rolündeki kullanıcılar daha sınırlı işlemler gerçekleştirebilmektedir. Bu sayede herkesin yalnızca kendi görev alanına uygun işlemleri yapması hedeflenmiştir.

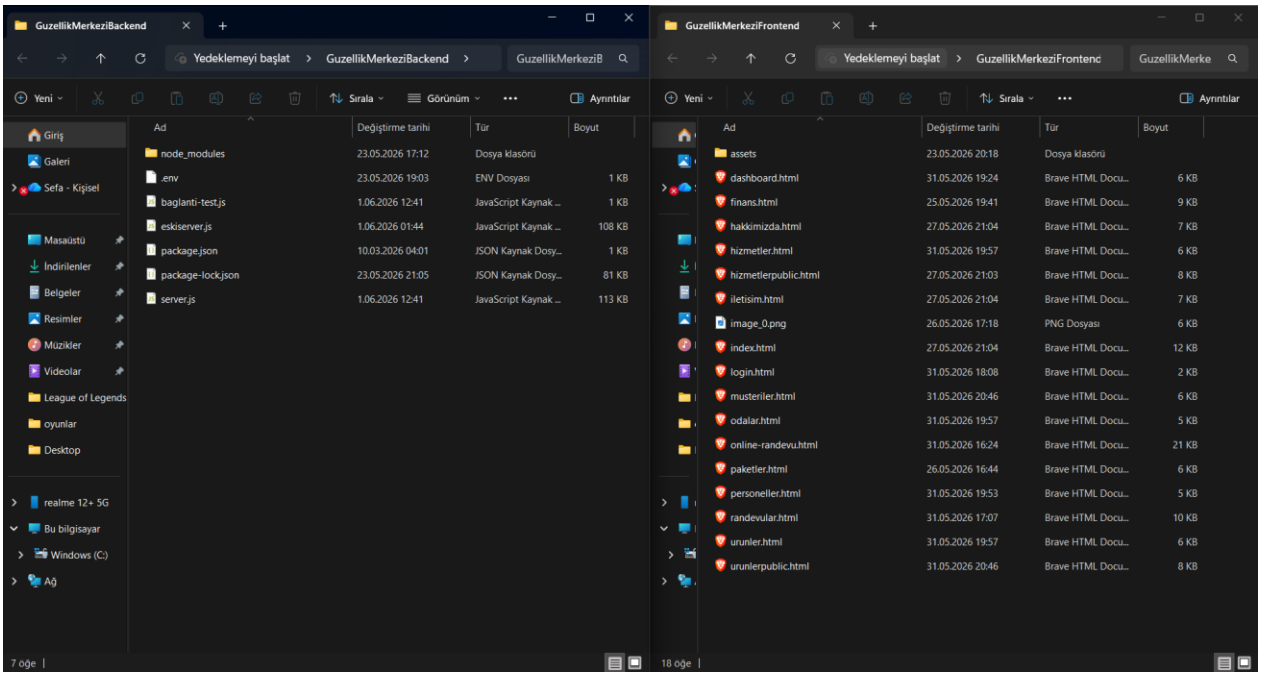
Sonuç olarak ShineBeauty projesinin amacı; güzellik merkezlerinin randevu, müşteri, hizmet, oda, personel, paket, ürün, stok ve finans süreçlerini tek bir merkezden yönetebileceği, rol

bazlı güvenlik içeren, veri bütünlüğünü koruyan, işlem hatalarını azaltan ve işletme verimliliğini artıran kapsamlı bir web tabanlı yönetim sistemi geliştirmektir.

Şekil 1: Dashboard Ekranı



Şekil 2: Klasör Yapısı



2. KULLANILAN TEKNOLOJİLER VE MİMARİ ÇÖZÜMLER

ShineBeauty projesi, backend ve frontend tarafı birbirinden ayrılmış bir mimariyle geliştirilmiştir. Backend tarafı Node.js ve Express.js kullanılarak hazırlanmıştır. Node.js, JavaScript dilinin sunucu tarafında çalışmasını sağlayan bir çalışma ortamıdır. Express.js ise HTTP isteklerini karşılamak, API adreslerini oluşturmak ve frontend ile veritabanı arasındaki iletişimi düzenlemek için kullanılmıştır. Bu yapı sayesinde frontend sayfaları, backend tarafında oluşturulan API adreslerine istek göndererek gerekli verileri alabilmekte veya yeni kayıt işlemleri yapabilmektedir.

Projede veritabanı olarak Microsoft SQL Server kullanılmıştır. SQL Server, ilişkisel veritabanı yapısı sunduğu için müşteri, randevu, hizmet, personel, oda, ürün, paket ve finans tabloları arasında düzenli ilişkiler kurulmasına olanak sağlamaktadır. Backend tarafında SQL Server bağlantısı için mssql ve msnodesqlv8 paketleri kullanılmıştır. msnodesqlv8 kullanımı, Windows Authentication ile SQL Server'a bağlanmayı mümkün hale getirmiştir. Bu sayede proje, yerel geliştirme ortamında SQL Server Express üzerinde çalışabilecek şekilde hazırlanmıştır.

Backend tarafında kullanılan temel paketlerden biri bcrypt paketidir. Bu paket, kullanıcı şifrelerinin düz metin olarak saklanmasını engellemek için kullanılmıştır. Kullanıcı oluşturulurken şifre hashlenmekte, giriş yapılırken girilen şifre veritabanındaki hash değer ile karşılaştırılmaktadır. Böylece veritabanında doğrudan okunabilir şifre tutulmaması sağlanmıştır.

Projede crypto modülü de kullanılmıştır. Kullanıcı sisteme giriş yaptığında rastgele bir token üretilmekte ve bu token basit oturum sistemi içinde saklanmaktadır. Frontend tarafı daha sonra korumalı API isteklerinde bu token değerini Authorization başlığı içinde göndermektedir. Backend tarafındaki giriş kontrol middleware'i, gelen token değerini kontrol ederek kullanıcının sisteme giriş yapıp yapmadığını doğrulamaktadır.

CORS paketi, frontend ile backend arasında farklı adres veya portlardan istek yapılabilmesi için kullanılmıştır. Express uygulamasında JSON veri okuma aktif hale getirilmiş, böylece frontend tarafından gönderilen form verileri JSON formatında backend tarafına aktarılabilmiştir. PDFKit paketi ise ürün satışı gibi işlemlerden sonra PDF fatura veya çıktı üretmek amacıyla projeye eklenmiştir.

Frontend tarafında HTML, CSS ve JavaScript kullanılmıştır. Sayfalar klasik web yapısıyla hazırlanmış, her modül için ayrı HTML sayfaları ve bu sayfaların işlemlerini yöneten ayrı JavaScript dosyaları oluşturulmuştur. Örneğin müşteri işlemleri için musteriler.html ve musteriler.js, randevu işlemleri için randevular.html ve randevular.js, finans işlemleri için finans.html ve finans.js, ürün işlemleri için urunler.html ve urunler.js kullanılmıştır. Bu dosya ayrımı, projenin daha okunabilir ve modüler olmasını sağlamıştır.

Sistemin mimari yapısı genel olarak istemci-sunucu mantığına dayanmaktadır. Kullanıcı, frontend ekranı üzerinden bir işlem başlatır. Bu işlem JavaScript tarafından yakalanır ve ilgili API adresine istek gönderilir. Backend tarafı isteği alır, gerekiyorsa giriş ve rol kontrolünden geçirir, verileri doğrular, SQL Server üzerinde gerekli sorguyu çalıştırır ve sonucu JSON formatında frontend tarafına döndürür. Frontend ise gelen cevaba göre tabloyu günceller, kullanıcıya mesaj gösterir veya formu temizler.

Bu mimaride REST benzeri API yapısı kullanılmıştır. GET istekleri veri listeleme ve detay getirme işlemlerinde, POST istekleri yeni kayıt oluşturma işlemlerinde, PUT ve PATCH istekleri güncelleme veya durum değiştirme işlemlerinde kullanılmıştır. Örneğin müşteri listeleme için GET /api/musteriler, müşteri ekleme için POST /api/musteriler, müşteri güncelleme için PUT /api/musteriler/:id, randevu onaylama için PATCH /api/randevular/:id/onayla yapısı kullanılmıştır.

Projenin önemli mimari çözümlerinden biri, veritabanı işlemlerinde parametreli sorgular kullanılmasıdır. Backend tarafında SQL sorgularına doğrudan kullanıcıdan gelen veri eklenmemiş,

bunun yerine .input yapısı ile parametreler tanımlanmıştır. Bu yaklaşım hem verilerin daha kontrollü işlenmesini sağlamakta hem de SQL enjeksiyonu gibi güvenlik risklerini azaltmaktadır.

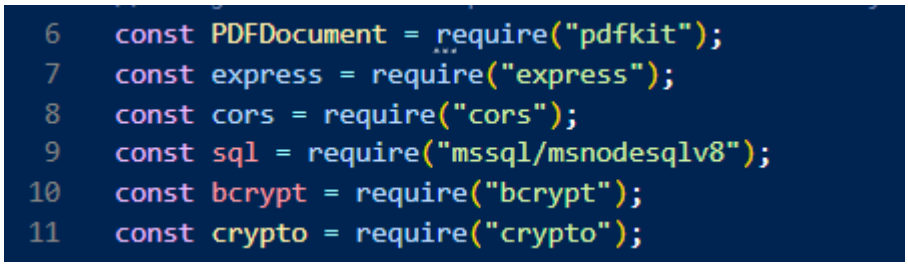
Bir diğer önemli mimari çözüm, bazı kritik işlemlerde transaction kullanılmasıdır. Özellikle paket satışı ve ürün stoğu ekleme gibi birden fazla tabloyu etkileyen işlemler transaction mantığı ile yürütülmüştür. Örneğin paket satışı yapılırken hem Paketler tablosuna kayıt eklenmekte hem de KasaHareketleri tablosuna gelir hareketi yazılmaktadır. Bu iki işlemden biri başarısız olursa transaction geri alınmakta ve yarım kayıt oluşması engellenmektedir. Aynı mantık ürün stoğu eklenirken ürün kaydı, gider kaydı ve kasa hareketinin birlikte yönetilmesi için de kullanılmıştır.

Randevu yönetimi tarafında da özel bir algoritmik kontrol uygulanmıştır. Randevu oluşturulurken seçilen hizmet seçeneğinin süresi alınmakta, başlangıç saatine göre bitiş saati hesaplanmakta, ardından seçilen personel ve oda için aynı zaman aralığında başka aktif veya bekleyen randevu olup olmadığı kontrol edilmektedir. Bu yapı, işletme içinde aynı oda veya personelin aynı anda iki farklı randevuya atanmasını engellemektedir.

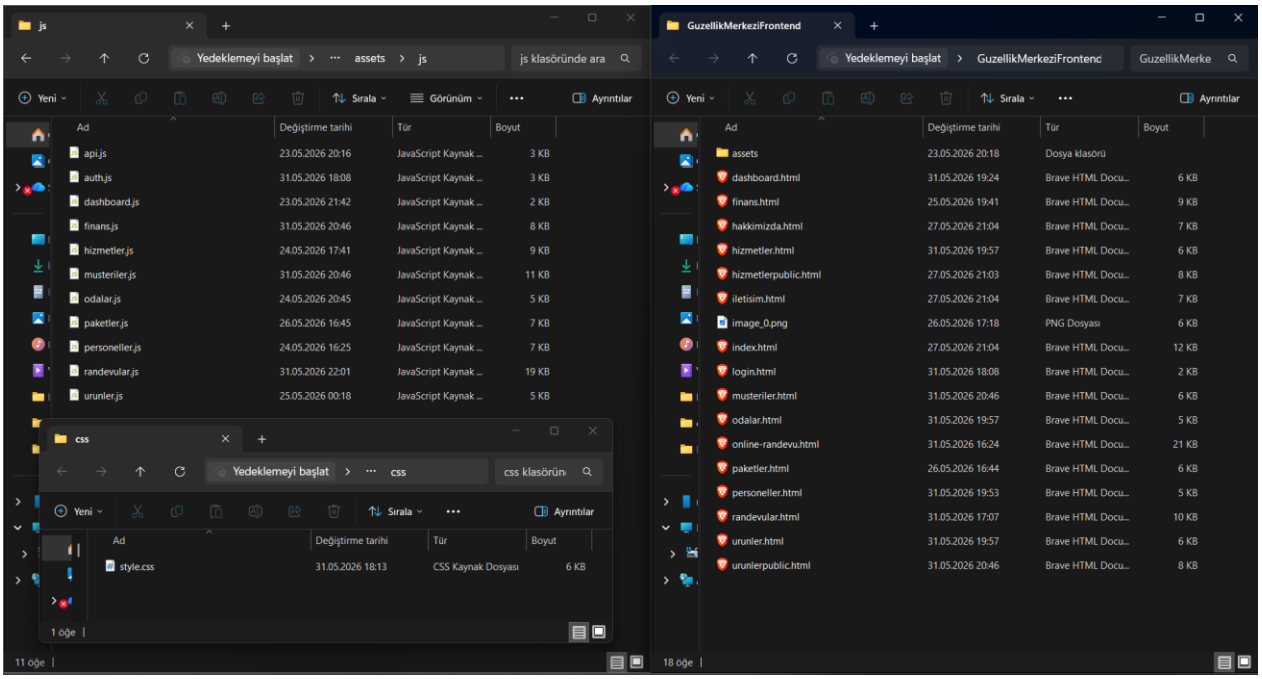
Şekil 1: Package.json



Şekil 2: Kullanılan Paketler



Şekil 3: Frontend Dosyaları



3. VERİ TABANI TASARIMI

ShineBeauty projesinin veritabanı tasarımı, güzellik merkezinin temel iş süreçlerini temsil eden tablolar üzerine kurulmuştur. Veritabanı ilişkisel yapıdadır ve tablolar arasında foreign key mantığına uygun bağlantılar bulunmaktadır. Sistemde müşteri, personel, hizmet, hizmet seçeneği, oda, randevu, ürün, ürün satışı, gider, kasa hareketi, paket ve paket kullanım kayıtları birbirleriyle ilişkili şekilde çalışmaktadır.

Veritabanındaki temel tablolardan biri Musteriler tablosudur. Bu tablo, güzellik merkezinden hizmet alan müşterilerin bilgilerini saklamak için kullanılır. Müşterinin ad, soyad, telefon, e-posta, cinsiyet, doğum tarihi, müşteri tipi, durum, T.C. kimlik numarası, notlar ve son randevu bilgileri bu tabloda tutulmaktadır. Randevu, paket ve ürün satışı gibi işlemler müşteri kaydıyla ilişkilendirildiği için Musteriler tablosu sistemin merkez tablolarından biridir.

Kullanicilar tablosu, sisteme giriş yapan admin ve personel kullanıcılarını saklamak için kullanılır. Bu tabloda kullanıcının ad, soyad, kullanıcı adı, şifre hash değeri, rol, durum ve kayıt

tarihi gibi bilgileri bulunur. Rol alanı, kullanıcının Admin mi yoksa Personel mi olduğunu belirler. Randevularda görevli personel bilgisi de bu tablo üzerinden takip edilmektedir. Bu nedenle Kullanicilar tablosu hem güvenlik hem de operasyonel randevu yönetimi açısından önemlidir.

Hizmetler tablosu, güzellik merkezinde sunulan ana hizmetleri tutar. Örneğin cilt bakımı, lazer, masaj veya benzeri hizmet başlıkları bu tabloda yer alabilir. Hizmetler tablosunda hizmet adı, hizmet tipi ve durum bilgisi bulunur. Tip alanı, hizmetin hangi oda tipiyle eşleşeceğini belirlemek için kullanılmaktadır. Böylece bir hizmet seçildiğinde, bu hizmete uygun oda tipleri sistem tarafından filtrelenebilmektedir.

HizmetSecenekleri tablosu, Hizmetler tablosuna bağlı alt hizmet seçeneklerini tutar. Bir ana hizmetin farklı süre ve fiyatlara sahip seçenekleri olabilir. Bu tabloda hizmet_id, seçenek adı, süre dakika bilgisi, fiyat ve durum alanları yer alır. Randevu oluşturulurken seçilen hizmet seçeneğinin süresi alınır ve randevunun bitiş saati bu süreye göre hesaplanır. Bu nedenle HizmetSecenekleri tablosu randevu planlama algoritmasının önemli bir parçasıdır.

Odalar tablosu, işletmedeki fiziksel oda veya işlem alanlarını temsil eder. Her odanın adı, tipi ve aktiflik durumu vardır. Hizmetler tablosundaki tip bilgisi ile Odalar tablosundaki tip bilgisi eşleştirilerek, seçilen hizmete uygun odalar listelenebilir. Randevu oluşturma sırasında oda çakışması kontrol edildiği için Odalar tablosu kaynak yönetimi açısından önemlidir.

Randevular tablosu, projenin en önemli tablolarından biridir. Bu tabloda müşterinin hangi hizmeti, hangi personelden, hangi odada, hangi tarih ve saat aralığında alacağı bilgisi tutulur. Randevular tablosu Musteriler, Kullanicilar, Hizmetler, HizmetSecenekleri ve Odalar tablolarına bağlıdır. Bu tabloda musteri_id, personel_id, hizmet_id, oda_id, hizmet_secenek_id, baslangic, bitis, durum, kaynak, müşteri notu ve iç not gibi alanlar kullanılır. Durum alanı randevunun Aktif, Bekliyor, Tamamlandi, Iptal veya Gelmedi gibi aşamalarını ifade eder. Kaynak alanı ise randevunun panelden mi yoksa web üzerinden mi oluşturulduğunu ayırt eder.

Urunler tablosu, güzellik merkezinde satılan ürünleri saklar. Bu tabloda ürün adı, kategori, stok miktarı, alış fiyatı, satış fiyatı, durum ve kayıt tarihi gibi alanlar yer alır. Ürün satış işlemlerinde stok miktarı otomatik olarak azaltıldığı için bu tablo stok yönetiminin merkezinde yer alır. Ayrıca stok miktarı belirli bir limitin altına düştüğünde dashboard üzerinde stok uyarısı oluşturulabilmektedir.

UrunSatislari tablosu, satılan ürünlerin geçmişini tutar. Bu tabloda müşteri, personel, ürün, adet, birim fiyat, toplam tutar, ödeme yöntemi, açıklama ve satış tarihi bilgileri bulunur. Ürün satışı yapıldığında bu tabloya satış kaydı eklenir, Urunler tablosundaki stok miktarı azaltılır ve KasaHareketleri tablosuna gelir kaydı işlenir. Böylece ürün satışı hem stok hem finans tarafına yansır.

Giderler tablosu, işletmenin gider kayıtlarını tutar. Giderin kategorisi, açıklaması, tutarı, ödeme yöntemi ve gider tarihi bu tabloda saklanır. Yeni bir gider eklendiğinde aynı zamanda KasaHareketleri tablosuna gider yönlü hareket de eklenmektedir. Bu yapı, giderlerin yalnızca liste olarak tutulmasını değil, finansal sonuçlara dahil edilmesini sağlar.

KasaHareketleri tablosu, sistemdeki gelir ve gider hareketlerinin merkezi kaydıdır. Ürün satışı, paket satışı, stok alımı veya manuel gider gibi işlemler bu tabloda gelir veya gider olarak tutulur. Bu tabloda işlem tipi, kategori, açıklama, tutar, tarih, ödeme yöntemi, ilgili tablo ve ilgili kayıt id bilgileri yer alabilir. Finans ekranındaki toplam gelir, toplam gider ve net kasa hesapları bu tablo üzerinden anlamlı hale gelmektedir.

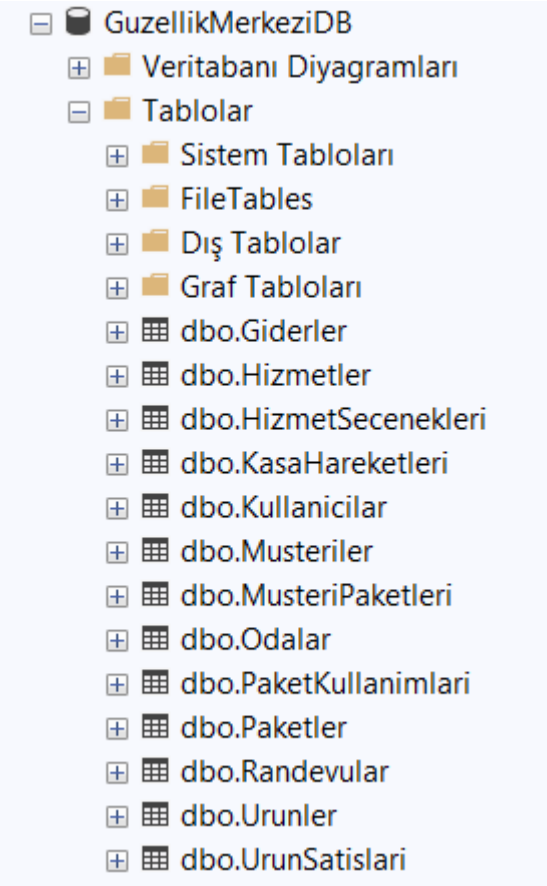
Paketler tablosu, müşterilere satılan seans paketlerini saklamak için kullanılır. Bir müşteri belirli bir hizmet veya hizmet seçeneği için paket satın aldığında toplam seans, kalan seans, fiyat, satış tarihi ve durum bilgisi bu tabloda tutulur. Randevu tamamlandığında müşterinin ilgili hizmet için aktif paketi varsa kalan seans sayısı azaltılabilir. Bu yapı, seans bazlı çalışan güzellik merkezi işletmeleri için temel bir ihtiyaçtır.

PaketKullanimlari tablosu, paket seanslarının hangi randevularda kullanıldığını kayıt altına alır. Bu tabloda paket id bilgisi, randevu id bilgisi, kullanılan seans sırası, tarih ve açıklama gibi bilgiler bulunur. Böylece bir paketin geçmişte hangi işlemler için kullanıldığı takip edilebilir.

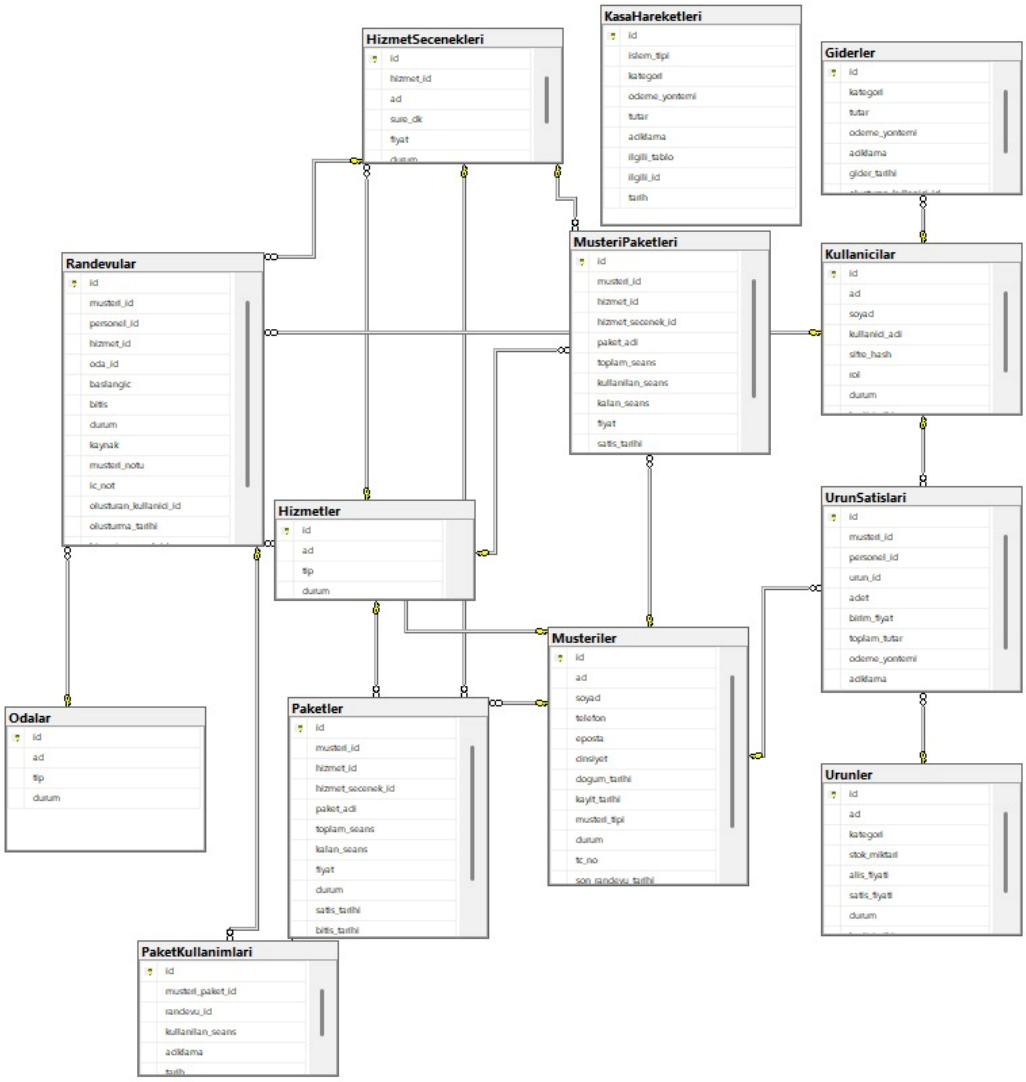
Veritabanındaki temel ilişkiler şu şekilde açıklanabilir: Bir müşteri birden fazla randevuya sahip olabilir. Bir personel birden fazla randevuya atanabilir. Bir hizmetin birden fazla hizmet seçeneği olabilir. Bir hizmet seçeneği birden fazla randevuda kullanılabilir. Bir oda birden fazla randevuda kullanılabilir; ancak aynı zaman aralığında çakışmaya izin verilmez. Bir müşteri birden fazla paket satın alabilir. Bir paket birden fazla kullanım kaydına sahip olabilir. Bir ürün birden fazla satış kaydında yer alabilir. Bir personel birden fazla ürün satışı gerçekleştirebilir. Kasa hareketleri ise satış, paket ve gider işlemlerinin finansal karşılığını saklar.

Bu veritabanı tasarımı sayesinde sistemdeki işlemler birbirinden kopuk şekilde değil, ilişkisel ve izlenebilir biçimde yönetilmektedir. Randevu kaydı yalnızca tarih bilgisi tutmaz; müşteri, personel, hizmet, hizmet seçeneği ve oda bilgileriyle birlikte anlam kazanır. Ürün satışı yalnızca bir satış kaydı değildir; aynı zamanda stok düşüşü ve kasa geliri oluşturur. Paket satışı yalnızca müşteri paketi oluşturmaz; aynı zamanda kasa hareketlerine gelir olarak yansır. Bu nedenle veritabanı yapısı, projenin bütün modüllerini birbirine bağlayan ana omurga görevini üstlenmektedir.

Şekil 1: Veritabanı Tabloları



Şekil 2: ER Diyagramı



Şekil 3: Randevu Oluşturma kodlarının bir kısmı

```
1373
1374  // Panelden yeni randevu oluşturma işlemi burada yapılır. Müşteri, personel, hizmet, oda ve saat bilgileri birlikte kontrol edilir.
1375  app.post("/api/randevular", girisGerekli, async (req, res) => {
1376    try {
1377      const {
1378        musteri_id,
1379        personel_id,
1380        hizmet_id,
1381        oda_id,
1382        hizmet_secenek_id,
1383        baslangic,
1384        musteri_notu,
1385      } = req.body;
1386
1387      const musteriId = sayiCevir(musteri_id);
1388      const personelId = sayiCevir(personel_id);
1389      const hizmetId = sayiCevir(hizmet_id);
1390      const odaId = sayiCevir(oda_id);
1391      const hizmetSecenekId = sayiCevir(hizmet_secenek_id);
1392
1393      if (!musteriId || !personelId || !hizmetId || !hizmetSecenekId || !odaId || !baslangic) {
1394        return res.status(400).json({ basarili: false, mesaj: "Zorunlu alan eksik" });
1395      }
```

4. SİSTEM MODÜLLERİ VE KOD MİMARİSİ

İçerik ... ShineBeauty projesi modüler bir yapıda geliştirilmiştir. Her ana iş süreci ayrı bir modül olarak ele alınmış ve bu modüller backend API adresleri ile frontend ekranları üzerinden yönetilmiştir. Sistemde giriş ve yetkilendirme, personel yönetimi, müşteri yönetimi, hizmet yönetimi, oda yönetimi, randevu yönetimi, online randevu, ürün ve stok yönetimi, paket ve seans yönetimi, finans ve kasa yönetimi, dashboard ve raporlama gibi modüller bulunmaktadır.

Backend tarafında tüm API yapısı server.js dosyası içinde toplanmıştır. Dosyanın başlangıcında gerekli paketler projeye dahil edilmekte, Express uygulaması başlatılmakta, JSON veri okuma ve CORS ayarları yapılmakta, SQL Server bağlantısı tanımlanmakta ve yardımcı fonksiyonlar oluşturulmaktadır. Yardımcı fonksiyonlar arasında tarih kontrolü, tarih çevirme, zaman çakışması kontrolü, id değerini sayıya çevirme ve aktiflik kontrolü gibi işlemler yer almaktadır. Bu fonksiyonlar, farklı API adreslerinde tekrar eden kontrolleri tek bir mantık altında toplamaktadır.

Giriş modülü, sistemin güvenlik başlangıç noktasıdır. Kullanıcı, login.html ekranı üzerinden kullanıcı adı ve şifre girerek sisteme giriş yapar. Frontend tarafı bu bilgileri /api/login adresine gönderir. Backend tarafında kullanıcı adı Kullanicilar tablosunda aranır. Kullanıcı bulunursa girilen şifre bcrypt ile veritabanındaki şifre hash değeriyle karşılaştırılır. Şifre doğruysa crypto modülü ile rastgele bir token üretilir ve kullanıcı bilgisi oturumlar nesnesine kaydedilir. Bu token frontend tarafında saklanır ve sonraki korumalı isteklerde Authorization başlığı ile gönderilir.

Personel yönetimi modülü, admin yetkisine bağlı olarak çalışır. Bu modülde yeni personel ekleme, personelleri listeleme, personel detayını getirme, personel bilgilerini güncelleme, personeli pasif yapma ve tekrar aktif hale getirme işlemleri bulunmaktadır. Personel kayıtları Kullanicilar tablosunda tutulur. Bu tabloda rol alanı Admin veya Personel ayrımını sağlar. Kullanıcı adı benzersizliği kontrol edilerek aynı kullanıcı adının birden fazla kişiye verilmesi engellenir.

Müşteri yönetimi modülü, müşterilerin sisteme kaydedilmesi ve düzenli şekilde takip edilmesi için oluşturulmuştur. Musteriler tablosu üzerinden müşteri ekleme, listeleme, arama, detay görüntüleme, güncelleme ve pasif hale getirme işlemleri yapılmaktadır. Müşteri kayıtlarında telefon, e-posta, müşteri tipi, T.C. kimlik numarası ve notlar gibi bilgiler tutulabildiği için işletme müşterilerini daha detaylı takip edebilmektedir. Ayrıca müşterinin geçmiş randevularını listeleyen API yapısı sayesinde müşteri kartı mantığı oluşturulmuştur.

Hizmet yönetimi modülü, işletmede verilen hizmetleri ve bu hizmetlerin alt seçeneklerini yönetir. Hizmetler tablosunda ana hizmetler tutulurken, HizmetSecenekleri tablosunda bu hizmetlere ait süre ve fiyat seçenekleri tutulur. Örneğin bir hizmetin 30 dakikalık, 60 dakikalık veya farklı fiyatlara sahip seçenekleri olabilir. Hizmet seçeneği randevu süresini doğrudan etkilediği için bu modül randevu sistemiyle birlikte çalışmaktadır.

Oda yönetimi modülü, işletmedeki işlem odalarının kaydedilmesini sağlar. Odalar tablosunda her odanın adı, tipi ve durumu tutulur. Hizmetlerde bulunan tip alanı ile odalardaki tip alanı eşleştirilerek, seçilen hizmete uygun odalar listelenir. Bu yapı, her hizmetin uygun fiziksel kaynakla eşleştirilmesini sağlamaktadır.

Randevu yönetimi modülü, projenin en kapsamlı modüllerinden biridir. Panel üzerinden randevu oluşturulurken müşteri, personel, hizmet, hizmet seçeneği, oda, başlangıç tarihi ve müşteri notu alınır. Backend tarafında önce hizmet seçeneğinin süresi bulunur, ardından başlangıç saatine bu süre eklenerek bitiş saati hesaplanır. Daha sonra aynı oda, aynı personel ve aynı müşteri için çakışan randevu olup olmadığı kontrol edilir. Eğer çakışma yoksa randevu Aktif durumunda oluşturulur. Bu yapı, manuel randevu sistemlerinde sık görülen saat çakışması hatalarını önlemeyi amaçlar.

Randevu modülünde yalnızca randevu oluşturma değil, randevu durumlarını yönetme işlemleri de bulunmaktadır. Randevu iptal edilebilir, tamamlanabilir, gelmedi olarak işaretlenebilir, başka bir tarihe taşınabilir veya bekleyen web randevusu onaylanabilir. Randevu tamamlandığında

müşterinin ilgili hizmet için paketi varsa paket seansından düşme işlemi yapılabilmektedir. Böylece randevu sistemi ile paket yönetimi doğrudan bağlantılı çalışmaktadır.

Online randevu modülü, web sitesi üzerinden müşteri tarafından randevu talebi oluşturulmasını sağlar. Bu modül giriş gerektirmeyen public API adresleri üzerinden çalışmaktadır. Müşteri hizmeti, hizmet seçeneğini, personeli ve tarih saat bilgisini seçerek randevu talebi gönderebilir. Ancak sistem güvenlik ve işletme kontrolü için bazı kontroller yapar. Telefon numarası 0 ile başlamalı ve 11 haneli olmalıdır. T.C. kimlik numarası girilecekse 11 haneli olmalıdır. Geçmiş tarihe randevu oluşturulamaz. Ayrıca sistemde bu telefon numarasına ait müşteri kaydı ve ilgili hizmet için aktif paket bulunmalıdır. Bu kontroller sağlanırsa randevu Bekliyor durumunda oluşturulur ve daha sonra panelden onaylanır.

Ürün ve stok yönetimi modülü, işletmede satılan ürünleri takip etmek için geliştirilmiştir. Ürünler tablosunda ürün adı, stok miktarı, alış fiyatı, satış fiyatı ve durum bilgisi tutulur. Yeni ürün eklendiğinde stok miktarı kaydedilir. Ürün satıldığında UrunSatislari tablosuna satış kaydı eklenir, Urunler tablosundaki stok miktarı azaltılır ve KasaHareketleri tablosuna gelir kaydı oluşturulur. Bu yapı sayesinde ürün satışı hem stok hem finans modülünü etkiler.

Paket ve seans yönetimi modülü, müşterilere satılan hizmet paketlerini takip eder. Paket satışı sırasında müşteri, hizmet, hizmet seçeneği, paket adı, toplam seans ve fiyat bilgileri alınır. Paket kaydı oluşturulduğunda toplam seans ve kalan seans bilgisi aynı değerle başlatılır. Paket satışı aynı zamanda kasaya gelir olarak işlenir. Randevu tamamlandığında ilgili paket bulunursa kalan seans bir azaltılır ve PaketKullanimlari tablosuna kullanım kaydı yazılır. Böylece hangi paketten hangi randevuda seans düşüldüğü takip edilebilir.

Finans ve kasa yönetimi modülü, sistemdeki gelir ve giderleri izlemek için oluşturulmuştur. Ürün satışı, paket satışı ve gider kayıtları finans ekranında takip edilebilir. KasaHareketleri tablosu üzerinden toplam gelir, toplam gider ve net kasa hesaplanabilir. Ayrıca gider ekleme işlemi

sırasında hem Giderler tablosuna kayıt yapılır hem de KasaHareketleri tablosuna gider hareketi işlenir. Bu sayede finansal veriler daha tutarlı şekilde saklanır.

Dashboard ve raporlama modülü, işletmenin genel durumunu özetleyen ekranları içerir. Dashboard üzerinde toplam müşteri sayısı, aktif randevu sayısı, bekleyen randevu sayısı, bugünkü randevular, tamamlanan randevular, iptal edilen randevular ve gelmeyen müşteri sayıları gibi bilgiler gösterilebilir. Raporlama tarafında genel ciro, günlük ciro, aylık ciro, en çok satan ürünler, en çok gelen müşteriler ve personel performansı gibi bilgiler sorgulanmaktadır. Bu yapı işletme sahibinin veriye dayalı karar almasını kolaylaştırmaktadır.

Fatura veya çıktı üretimi için ürün satışına bağlı PDF üretim mantığı da projede yer almaktadır. PDFKit kullanılarak ürün satış bilgileri, müşteri, personel, ürün adı, adet, birim fiyat, toplam tutar ve ödeme yöntemi gibi bilgiler PDF formatında hazırlanabilmektedir. Bu özellik, ileride e-fatura veya satış fişi benzeri yapılara temel oluşturabilecek niteliktedir.

Frontend mimarisi ise her modül için ayrı sayfa ve ayrı JavaScript dosyası mantığıyla hazırlanmıştır. dashboard.html yönetim paneli ekranını, musteriler.html müşteri işlemlerini, randevular.html randevu işlemlerini, hizmetler.html hizmet işlemlerini, odalar.html oda işlemlerini, personeller.html personel işlemlerini, urunler.html ürün ve stok işlemlerini, paketler.html paket yönetimini, finans.html ise kasa ve finans ekranını temsil eder. Public tarafta index.html, hakkimizda.html, iletisim.html, hizmetlerpublic.html, urunlerpublic.html ve online-randevu.html sayfaları yer almaktadır. Böylece yönetim paneli ile müşteri tarafı birbirinden ayrılmıştır.

Şekil 1: girisGerekli Fonksiyonu

```
function girisGerekli(req, res, next) {
  const auth = req.headers.authorization;

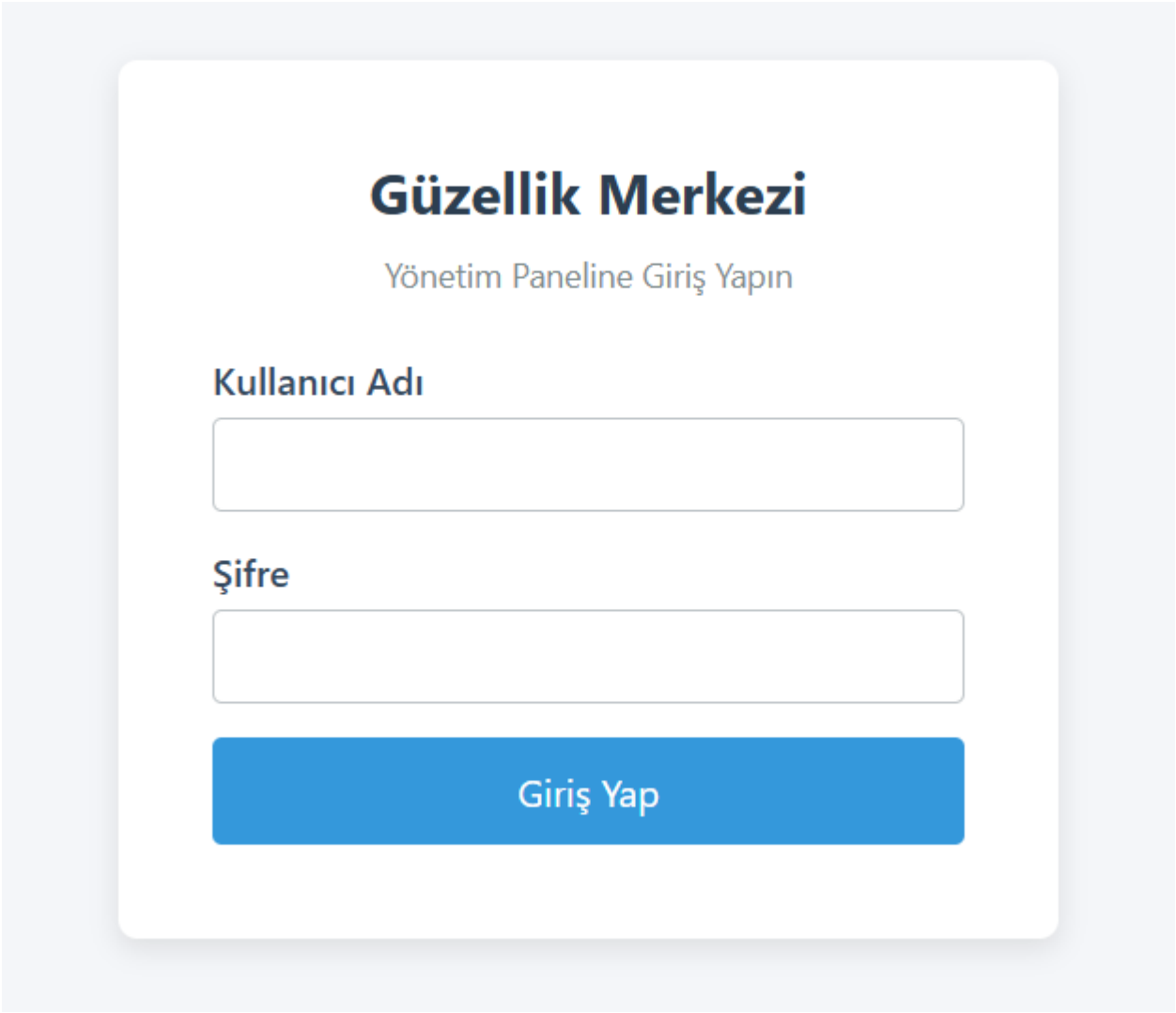
  if (!auth || !auth.startsWith("Bearer ")) {
    return res.status(401).json({
      basarili: false,
      mesaj: "Token yok. Once giris yapmalisin.",
    });
  }

  const token = auth.split(" ")[1];

  if (!oturumlar[token]) {
    return res.status(401).json({
      basarili: false,
      mesaj: "Token gecersiz. Tekrar giris yap.",
    });
  }

  req.kullanici = oturumlar[token];
  next();
}
```

Şekil 2: login.html Tasarımı



Şekil 3: Giriş işleminin sorgulandığı kod bloğu

```
app.post("/api/login", async (req, res) => {
  try {
    const { kullanici_adi, sifre } = req.body;

    if (!kullanici_adi || !sifre) {
      return res.status(400).json({
        basarili: false,
        mesaj: "kullanici_adi ve sifre zorunludur",
      });
    }

    const sonuc = await havuz
      .request()
      .input("kullanici_adi", sql.NVarChar(100), kullanici_adi)
      .query(`
        SELECT TOP 1 *
        FROM dbo.Kullanicilar
        WHERE kullanici_adi = @kullanici_adi
      `);

    if (sonuc.recordset.length === 0) {
      return res.status(401).json({
        basarili: false,
        mesaj: "Kullanici adi veya sifre hatali",
      });
    }

    const kullanici = sonuc.recordset[0];

    if (!bitAktifMi(kullanici.durum)) {
      return res.status(403).json({
        basarili: false,
        mesaj: "Bu hesap pasif durumda",
      });
    }
```

Şekil 4: Panel Üzerinden Randevu Oluşturma Kodları

```
app.post("/api/randevular", girisGerekli, async (req, res) => {
  try {
    const {
      musteri_id,
      personel_id,
      hizmet_id,
      oda_id,
      hizmet_secenek_id,
      baslangic,
      musteri_notu,
    } = req.body;

    const musteriId = sayiCevir(musteri_id);
    const personelId = sayiCevir(personel_id);
    const hizmetId = sayiCevir(hizmet_id);
    const odaId = sayiCevir(oda_id);
    const hizmetSecenekId = sayiCevir(hizmet_secenek_id);

    if (!musteriId || !personelId || !hizmetId || !hizmetSecenekId || !odaId || !baslangic) {
      return res.status(400).json({ basarili: false, mesaj: "Zorunlu alan eksik" });
    }


    // Frontend tarih bilgisini string gönderir. Burada Date nesnesine çevrilip geçerli tarih olup olmadığı kontrol edilir.
    const baslangicTarihi = new Date(baslangic);
    if (isNaN(baslangicTarihi.getTime())) {
      return res.status(400).json({ basarili: false, mesaj: "Geçersiz başlangıç tarihi" });
    }

    // Geçmiş bir tarih veya saate randevu verilmemesi için kayıt işleminden önce kontrol yapılır.
    if (baslangicTarihi < new Date()) {
  return res.status(400).json({
    basarili: false,
    mesaj: "Geçmiş bir tarihe randevu oluşturamazsınız!"
  });
}
```

```
    // Seçilen hizmet seçeneğinin süresi veritabanından alınır. Randevunun bitiş saati bu süreye göre hesaplanır.
    const secenekSonuc = await havuz.request()
      .input("hizmet_secenek_id", sql.Int, hizmetSecenekId)
      .query(`
        SELECT hs.sure_dk, hs.hizmet_id
        FROM dbo.HizmetSecenekleri hs
        WHERE hs.id = @hizmet_secenek_id AND hs.durum = 1
      `);

    if (secenekSonuc.recordset.length === 0) {
      return res.status(400).json({ basarili: false, mesaj: "Hizmet seçeneği bulunamadı" });
    }

    const secenek = secenekSonuc.recordset[0];

    // Başlangıç saatine hizmet süresi eklenerek randevunun bitiş saati hesaplanır.
    const bitisTarihi = new Date(baslangicTarihi.getTime() + parseInt(secenek.sure_dk) * 60 * 1000);


    // Geçmiş bir tarih veya saate randevu verilmemesi için kayıt işleminden önce kontrol yapılır.
    if (baslangicTarihi < new Date()) {
        return res.status(400).json({ basarili: false, mesaj: "Geçmiş bir tarihe randevu oluşturamazsınız!" });
    }
```

```
// Randevu oluşturulmadan önce müşterinin seçilen hizmet için kullanılabilir seansı olup olmadığı kontrol edilir.
const paketKontrol = await havuz.request()
    .input("musteri_id", sql.Int, musteriId)
    .input("hizmet_id", sql.Int, hizmetId)
    .query(`
        SELECT ISNULL(SUM(kalan_seans), 0) AS toplam_kalan
        FROM dbo.Paketler
        WHERE musteri_id = @musteri_id
        AND hizmet_id = @hizmet_id
        AND kalan_seans > 0
    `);

if (paketKontrol.recordset[0].toplam_kalan <= 0) {
    return res.status(400).json({
        basarili: false,
        mesaj: "İşlem Reddedildi: Bu müşterinin seçilen hizmet için kullanılabilir (kalan) seansı bulunmamaktadır!"
    });
}
```

```
// Aynı odada aynı zaman aralığına denk gelen aktif veya bekleyen randevu varsa yeni kayıt engellenir.
const odaCakisma = await havuz.request()
  .input("oda_id", sql.Int, odaId)
  .input("baslangic", sql.DateTime2, baslangicTarihi)
  .input("bitis", sql.DateTime2, bitisTarihi)
  .query(`
    SELECT TOP 1 id FROM dbo.Randevular
    WHERE oda_id = @oda_id AND durum IN ('Aktif', 'Bekliyor')
      AND @baslangic < bitis AND @bitis > baslangic
  `);

if (odaCakisma.recordset.length > 0) {
  return res.status(409).json({ basarili: false, mesaj: "Seçtiğiniz oda bu saat aralığında zaten dolu." });
}


// Seçilen personelin aynı saat aralığında başka randevusu olup olmadığı kontrol edilir.
const personelCakisma = await havuz.request()
  .input("personel_id", sql.Int, personelId)
  .input("baslangic", sql.DateTime2, baslangicTarihi)
  .input("bitis", sql.DateTime2, bitisTarihi)
  .query(`
    SELECT TOP 1 id FROM dbo.Randevular
    WHERE personel_id = @personel_id AND durum IN ('Aktif', 'Bekliyor')
      AND @baslangic < bitis AND @bitis > baslangic
  `);

if (personelCakisma.recordset.length > 0) {
  return res.status(409).json({ basarili: false, mesaj: "Bu personelin bu saatte başka randevusu var." });
}
```

```
// Aynı müşteriye aynı hizmet için aynı zaman aralığında tekrar randevu verilmesi engellenir.
const musteriCakisma = await havuz.request()
  .input("musteri_id", sql.Int, musteriId)
  .input("hizmet_id", sql.Int, hizmetId)
  .input("baslangic", sql.DateTime2, baslangicTarihi)
  .input("bitis", sql.DateTime2, bitisTarihi)
  .query(`
    SELECT TOP 1 id FROM dbo.Randevular
    WHERE musteri_id = @musteri_id AND hizmet_id = @hizmet_id AND durum IN ('Aktif', 'Bekliyor')
      AND @baslangic < bitis AND @bitis > baslangic
  `);

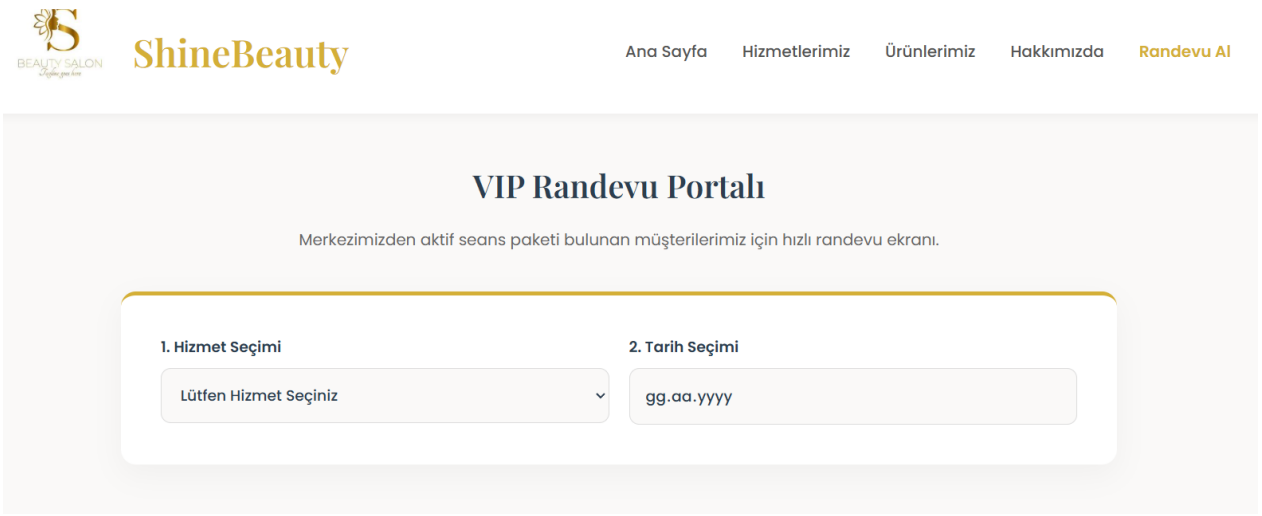
if (musteriCakisma.recordset.length > 0) {
  return res.status(409).json({ basarili: false, mesaj: "Aynı müşteri için aynı saatlerde aynı hizmet zaten var." });
}
```

```
  const sonuc = await havuz.request()
    .input("musteri_id", sql.Int, musteriId)
    .input("personel_id", sql.Int, personelId)
    .input("hizmet_id", sql.Int, hizmetId)
    .input("oda_id", sql.Int, odaId)
    .input("baslangic", sql.DateTime2, baslangicTarihi)
    .input("bitis", sql.DateTime2, bitisTarihi)
    .input("musteri_notu", sql.NVarChar(1000), musteri_notu || null)
    .input("olusturan_kullanici_id", sql.Int, req.kullanici.kullanici_id)
    .input("hizmet_secenek_id", sql.Int, hizmetSecenekId)
    .query(`
      INSERT INTO dbo.Randevular
      (musteri_id, personel_id, hizmet_id, oda_id, baslangic, bitis, durum, kaynak, musteri_notu, olusturan_kullanici_id, hizmet_secenek_id)
      OUTPUT INSERTED.*
      VALUES
      (@musteri_id, @personel_id, @hizmet_id, @oda_id, @baslangic, @bitis, 'Aktif', 'Panel', @musteri_notu, @olusturan_kullanici_id, @hizmet_secenek_id)
    `);

  await havuz.request()
    .input("musteri_id", sql.Int, musteriId)
    .input("son_randevu_tarihi", sql.DateTime2, baslangicTarihi)
    .query(`UPDATE dbo.Musteriler SET son_randevu_tarihi = @son_randevu_tarihi WHERE id = @musteri_id`);

  res.status(201).json({ basarili: true, mesaj: "Randevu başarıyla eklendi", veri: sonuc.recordset[0] });
} catch (hata) {
  res.status(500).json({ basarili: false, mesaj: "Randevu eklenemedi", hata: hata.message });
}
});
```

Şekil 5: Web Üzerinden Randevu Oluşturma Ekranı Ve Kod Bloğu



```
// Web sitesinden gelen randevu talepleri burada oluşturulur. Bu kayıtlar doğrudan aktif yapılmaz, Bekliyor durumunda kalır.
app.post("/api/public/randevular", async (req, res) => {
  try {
    const { ad, soyad, telefon, tc_no, hizmet_id, hizmet_secenek_id, personel_id, tarihSaat } = req.body;


    if (!telefon || telefon.length !== 11 || !telefon.startsWith("0")) {
        return res.status(400).json({ basarili: false, mesaj: "Telefon numarası 0 ile başlamalı ve tam 11 haneli olmalıdır." });
    }
    if (tc_no && tc_no.trim() !== "" && tc_no.length !== 11) {
        return res.status(400).json({ basarili: false, mesaj: "TC Kimlik Numarası girilecekse tam 11 haneli olmalıdır." });
    }


    const baslangicTarihi = new Date(tarihSaat);
    if (baslangicTarihi < new Date()) {
        return res.status(400).json({ basarili: false, mesaj: "Geçmiş bir tarihe randevu oluşturamazsınız." });
    }


    const musteriSonuc = await havuz.request()
      .input("telefon", sql.NVarChar(20), telefon)
      .query("SELECT id FROM dbo.Musteriler WHERE telefon = @telefon");

    if (musteriSonuc.recordset.length === 0) {
        return res.status(400).json({
            basarili: false,
            mesaj: "Sistemimizde bu telefon numarasıyla eşleşen bir kayıt bulunamadı. Lütfen önce merkezimizle iletişime geçiniz."
        });
    }
    const musteriId = musteriSonuc.recordset[0].id;
```

```
    const paketKontrol = await havuz.request()
        .input("musteri_id", sql.Int, musteriId)
        .input("hizmet_id", sql.Int, hizmet_id)
        .query(`
            SELECT ISNULL(SUM(kalan_seans), 0) AS toplam_kalan
            FROM dbo.Paketler
            WHERE musteri_id = @musteri_id
            AND hizmet_id = @hizmet_id
            AND kalan_seans > 0
        `);

    if (paketKontrol.recordset[0].toplam_kalan <= 0) {
        return res.status(400).json({
            basarili: false,
            mesaj: "İşlem Reddedildi: Seçilen hizmet için aktif bir seans paketiniz bulunmamaktadır!"
        });
    }
```

```
    // Seçilen hizmet seçeneğinin süresi veritabanından alınır. Randevunun bitiş saati bu süreye göre hesaplanır.
    const secenekSonuc = await havuz.request()
      .input("id", sql.Int, hizmet_secenek_id)
      .query("SELECT sure_dk FROM dbo.HizmetSecenekleri WHERE id = @id");
    const sureDk = secenekSonuc.recordset.length > 0 ? secenekSonuc.recordset[0].sure_dk : 30;

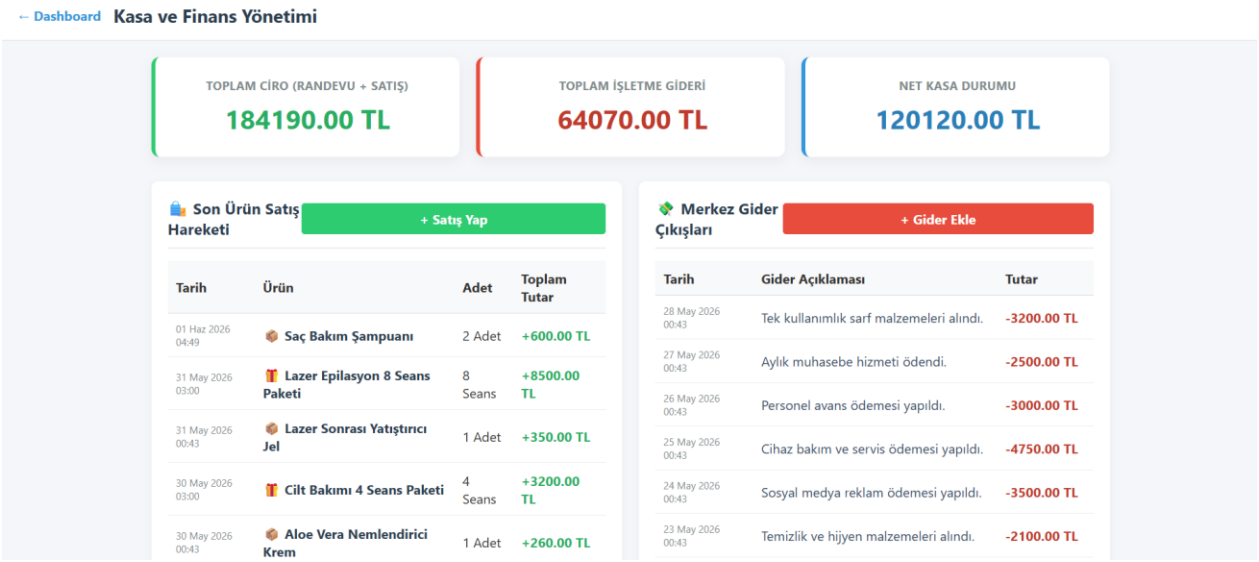
    const bitisTarihi = new Date(baslangicTarihi.getTime() + sureDk * 60000);


    const odaSonuc = await havuz.request().query("SELECT TOP 1 id FROM dbo.Odalar WHERE durum = 1");
    const odaId = odaSonuc.recordset.length > 0 ? odaSonuc.recordset[0].id : 1;


    await havuz.request()
      .input("musteri_id", sql.Int, musteriId)
      .input("personel_id", sql.Int, personel_id)
      .input("hizmet_id", sql.Int, hizmet_id)
      .input("oda_id", sql.Int, odaId)
      .input("hizmet_secenek_id", sql.Int, hizmet_secenek_id)
      .input("baslangic", sql.DateTime, baslangicTarihi)
      .input("bitis", sql.DateTime, bitisTarihi)
      .query(`
        INSERT INTO dbo.Randevular (musteri_id, personel_id, hizmet_id, oda_id, hizmet_secenek_id, baslangic, bitis, durum, kaynak)
        VALUES (@musteri_id, @personel_id, @hizmet_id, @oda_id, @hizmet_secenek_id, @baslangic, @bitis, 'Bekliyor', 'Web')
      `);

    res.json({ basarili: true, mesaj: "Randevu talebiniz başarıyla alındı! Merkezimiz onay için sizinle iletişime geçecektir." });
  } catch (hata) {
    res.status(500).json({ basarili: false, mesaj: "Randevu oluşturulamadı", hata: hata.message });
  }
});
```

Şekil 6: finans.html Sayfasının Tasarımı



Şekil 7: finans.html Sayfasında Gelirlerin Hesaplandığı Kod Bloğu

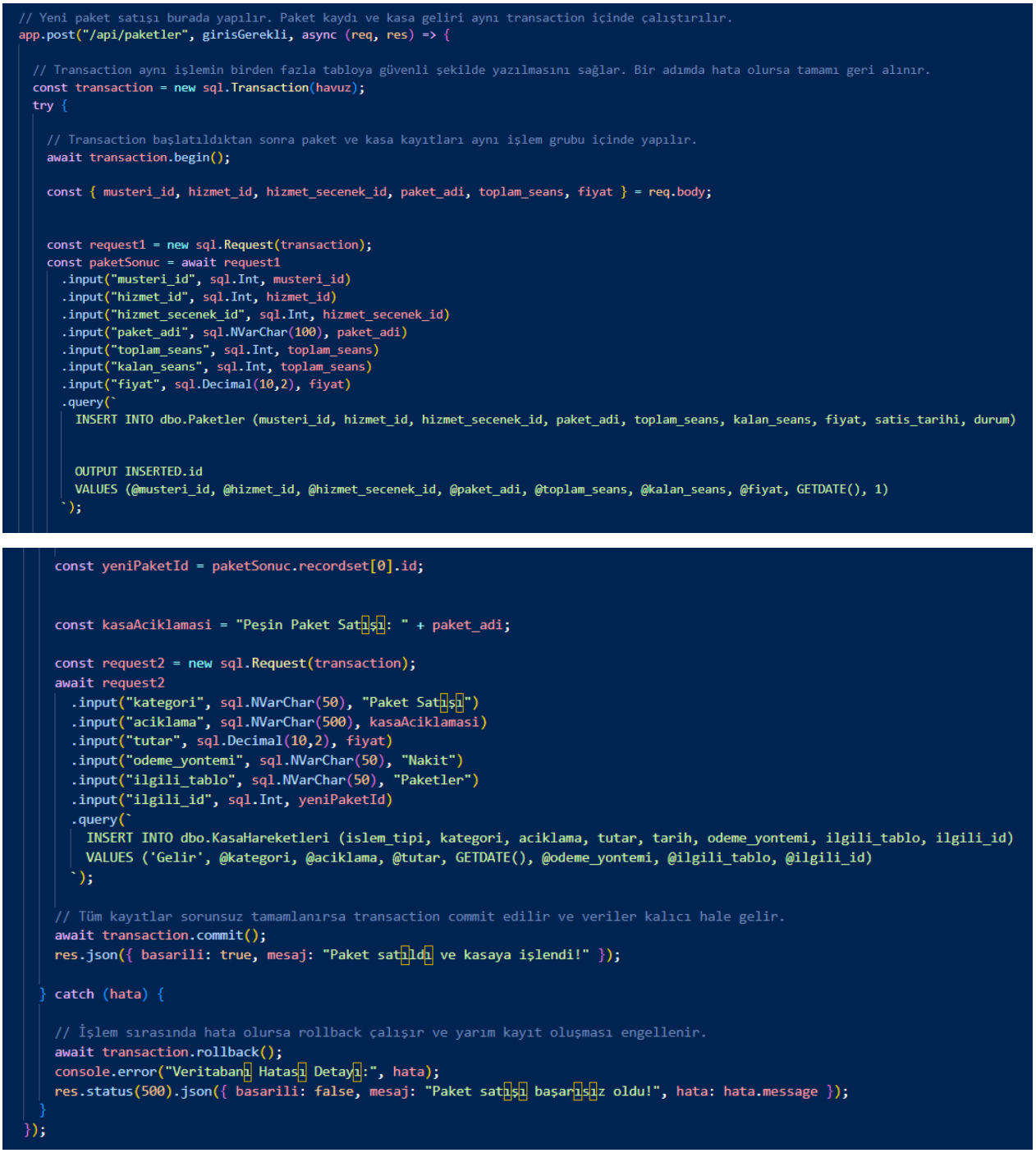
```
// Finans ekranındaki toplam gelir, gider ve net kasa özeti bu route ile hazırlanır.
app.get("/api/finans/ozet", girisGerekli, async (req, res) => {
  try {
    const sonuc = await havuz.request().query(`
      SELECT
        ISNULL((SELECT SUM(tutar) FROM dbo.KasaHareketleri WHERE islem_tipi = 'Gelir' AND kategori = 'Randevu'), 0) AS randevu_geliri,
        ISNULL((SELECT SUM(tutar) FROM dbo.KasaHareketleri WHERE islem_tipi = 'Gelir' AND kategori = 'Ürün Satışı'), 0) AS urun_geliri,
        ISNULL((SELECT SUM(tutar) FROM dbo.KasaHareketleri WHERE islem_tipi = 'Gelir' AND kategori = 'Paket Satışı'), 0) AS paket_geliri,
        ISNULL((SELECT SUM(tutar) FROM dbo.Giderler), 0) AS toplam_gider
    `);

    const veri = sonuc.recordset[0];


    const toplamGelir = veri.randevu_geliri + veri.urun_geliri + veri.paket_geliri;
    const netKasa = toplamGelir - veri.toplam_gider;

    res.json({
      basarili: true,
      ozet: {
        randevu_geliri: veri.randevu_geliri,
        urun_geliri: veri.urun_geliri,
        paket_geliri: veri.paket_geliri,
        toplam_gelir: toplamGelir,
        toplam_gider: veri.toplam_gider,
        net_kasa: netKasa
      }
    });
  } catch (hata) {
    res.status(500).json({ basarili: false, mesaj: "Finans özeti alınamadı", hata: hata.message });
  }
});
```

Şekil 8: Paket Satışının Sağlandığı Kod Bloğu



```
// Yeni paket satışı burada yapılır. Paket kaydı ve kasa geliri aynı transaction içinde çalıştırılır.
app.post("/api/paketler", girisGerekli, async (req, res) => {

  // Transaction aynı işlemin birden fazla tabloya güvenli şekilde yazılmasını sağlar. Bir adımda hata olursa tamamı geri alınır.
  const transaction = new sql.Transaction(havuz);
  try {

    // Transaction başlatıldıktan sonra paket ve kasa kayıtları aynı işlem grubu içinde yapılır.
    await transaction.begin();

    const { musteri_id, hizmet_id, hizmet_secenek_id, paket_adi, toplam_seans, fiyat } = req.body;


    const request1 = new sql.Request(transaction);
    const paketSonuc = await request1
      .input("musteri_id", sql.Int, musteri_id)
      .input("hizmet_id", sql.Int, hizmet_id)
      .input("hizmet_secenek_id", sql.Int, hizmet_secenek_id)
      .input("paket_adi", sql.NVarChar(100), paket_adi)
      .input("toplam_seans", sql.Int, toplam_seans)
      .input("kalan_seans", sql.Int, toplam_seans)
      .input("fiyat", sql.Decimal(10,2), fiyat)
      .query(`
        INSERT INTO dbo.Paketler (musteri_id, hizmet_id, hizmet_secenek_id, paket_adi, toplam_seans, kalan_seans, fiyat, satis_tarihi, durum)


        OUTPUT INSERTED.id
        VALUES (@musteri_id, @hizmet_id, @hizmet_secenek_id, @paket_adi, @toplam_seans, @kalan_seans, @fiyat, GETDATE(), 1)
      `);
```

```
    const yeniPaketId = paketSonuc.recordset[0].id;


    const kasaAciklamasi = "Peşin Paket Satışı: " + paket_adi;

    const request2 = new sql.Request(transaction);
    await request2
      .input("kategori", sql.NVarChar(50), "Paket Satışı")
      .input("aciklama", sql.NVarChar(500), kasaAciklamasi)
      .input("tutar", sql.Decimal(10,2), fiyat)
      .input("odeme_yontemi", sql.NVarChar(50), "Nakit")
      .input("ilgili_tablo", sql.NVarChar(50), "Paketler")
      .input("ilgili_id", sql.Int, yeniPaketId)
      .query(`
        INSERT INTO dbo.KasaHareketleri (islem_tipi, kategori, aciklama, tutar, tarih, odeme_yontemi, ilgili_tablo, ilgili_id)
        VALUES ('Gelir', @kategori, @aciklama, @tutar, GETDATE(), @odeme_yontemi, @ilgili_tablo, @ilgili_id)
      `);

    // Tüm kayıtlar sorunsuz tamamlanırsa transaction commit edilir ve veriler kalıcı hale gelir.
    await transaction.commit();
    res.json({ basarili: true, mesaj: "Paket satıldı ve kasaya işlendi!" });

  } catch (hata) {

    // İşlem sırasında hata olursa rollback çalışır ve yarım kayıt oluşması engellenir.
    await transaction.rollback();
    console.error("Veritabanı Hatası Detayı:", hata);
    res.status(500).json({ basarili: false, mesaj: "Paket satışı başarısız oldu!", hata: hata.message });
  }
});
```

Şekil 9: Ürün Ekleme İşlemini Gerçekleştiren Ve Kasaya Gelir/Gider Olarak Ekleyen Kod Bloğu

```
// Yeni ürün eklendiğinde stok, alış ve satış fiyatı kaydedilir. Alış maliyeti gider olarak da işlenir.
app.post("/api/urunler", girisGerekli, sadeceAdmin, async (req, res) => {

  // Transaction aynı işlemin birden fazla tabloya güvenli şekilde yazılmasını sağlar. Bir adımda hata olursa tamamı geri alınır.
  const transaction = new sql.Transaction(havuz);
  try {
    await transaction.begin();

    const { ad, stok, alis_fiyati, fiyat, kategori } = req.body;
    const stokMiktari = sayiCevir(stok) || 0;
    const alisFiyati = parseFloat(alis_fiyati) || 0;
    const satisFiyati = parseFloat(fiyat) || 0;


    const request1 = new sql.Request(transaction);
    await request1
      .input("ad", sql.NVarChar, ad)
      .input("stok_miktari", sql.Int, stokMiktari)
      .input("alis_fiyati", sql.Decimal(10,2), alisFiyati)
      .input("satis_fiyati", sql.Decimal(10,2), satisFiyati)
      .query(`
        INSERT INTO dbo.Urunler (ad, stok_miktari, alis_fiyati, satis_fiyati, durum)
        VALUES (@ad, @stok_miktari, @alis_fiyati, @satis_fiyati, 1)
      `);


    const toplamMaliyet = alisFiyati * stokMiktari;


    const aciklama = `Stok Alımı: ${ad} (${stokMiktari} Adet)`;
    const request2 = new sql.Request(transaction);
    await request2
      .input("kategori", sql.NVarChar(50), kategori || "Stok Alımı")
      .input("aciklama", sql.NVarChar(500), aciklama)
      .input("tutar", sql.Decimal(10,2), toplamMaliyet)
      .query(`
        INSERT INTO dbo.Giderler (kategori, aciklama, tutar, odeme_yontemi, gider_tarihi)
        VALUES (@kategori, @aciklama, @tutar, 'Nakit', GETDATE())
      `);
```

```
    const request3 = new sql.Request(transaction);
    await request3
      .input("tutar", sql.Decimal(10,2), toplamMaliyet)
      .input("aciklama", sql.NVarChar(500), "Stok Alım Gideri: " + aciklama)
      .query(`
        INSERT INTO dbo.KasaHareketleri (islem_tipi, kategori, aciklama, tutar, tarih, odeme_yontemi)
        VALUES ('Gider', 'Stok Alımı', @aciklama, @tutar, GETDATE(), 'Nakit')
      `);

    // Tüm kayıtlar sorunsuz tamamlanırsa transaction commit edilir ve veriler kalıcı hale gelir.
    await transaction.commit();
    res.json({ basarili: true, mesaj: "Ürün eklendi, maliyet kasadan gider olarak düşüldü!" });

  } catch (hata) {

    // İşlem sırasında hata olursa rollback çalışır ve yarım kayıt oluşması engellenir.
    await transaction.rollback();
    console.error("Ürün ekleme hatası:", hata);
    res.status(500).json({ basarili: false, mesaj: "Ürün ekleme işlemi başarısız!", hata: hata.message });
  }
});
```

5. GÜVENLİK VE YETKİLENDİRME

ShineBeauty projesinde güvenlik ve yetkilendirme yapısı, kullanıcı girişi, şifre güvenliği, token tabanlı oturum kontrolü, rol bazlı erişim ve veritabanı sorgu güvenliği üzerine kurulmuştur. Sistemde admin ve personel ayrımı bulunduğu için her kullanıcının aynı işlemleri yapmasına izin verilmemiştir. Böylece işletme içinde görev ve yetki sınırları korunmaya çalışılmıştır.

Kullanıcı şifreleri veritabanında düz metin olarak saklanmamaktadır. Yeni kullanıcı veya ilk admin oluşturulurken şifre bcrypt ile hashlenmekte ve Kullanicilar tablosunda sifre_hash alanına kaydedilmektedir. Kullanıcı giriş yaptığında sisteme girilen şifre tekrar hashlenmiş şekilde karşılaştırılmaz; bcrypt.compare fonksiyonu ile girilen şifrenin veritabanındaki hash değerine uyup uymadığı kontrol edilir. Bu yapı, veritabanına erişilse bile kullanıcı şifrelerinin doğrudan okunmasını engelleyen önemli bir güvenlik önlemidir.

Giriş işlemi başarılı olduğunda crypto modülü ile rastgele bir token oluşturulur. Bu token, backend tarafında oturumlar isimli nesne içinde saklanır. Frontend tarafı bu token değerini korumalı API isteklerinde Authorization başlığı içinde Bearer token formatında gönderir. Backend tarafındaki girisGerekli middleware'i, gelen istekte token olup olmadığını, token formatının doğru olup olmadığını ve token değerinin aktif oturumlar içinde bulunup bulunmadığını kontrol eder. Token yoksa veya geçersizse işlem 401 yetkisiz hatasıyla reddedilir.

Rol bazlı erişim için sadeceAdmin isimli middleware kullanılmıştır. Bu middleware, giriş yapan kullanıcının rol bilgisini kontrol eder. Eğer kullanıcının rolü Admin değilse işlem 403 yetki hatasıyla engellenir. Personel ekleme, personel düzenleme, hizmet ekleme, oda düzenleme, ürün durum değiştirme gibi yönetimsel işlemler admin yetkisine bağlanmıştır. Bu yapı, personel hesabı ile sisteme giren kullanıcıların tüm sistem ayarlarını değiştirmesini engeller.

Projede API adresleri iki gruba ayrılmıştır. Birinci grup, giriş gerektiren yönetim paneli API adresleridir. Bu adreslerde girisGerekli middleware'i kullanılarak oturum kontrolü yapılır. İkinci grup ise public API adresleridir. Public adresler hizmet listesi, ürün listesi ve online randevu talebi

gibi müşteri tarafında kullanılacak işlemler için oluşturulmuştur. Ancak public adreslerde de tamamen kontrolsüz işlem yapılmamaktadır. Örneğin online randevu oluştururken telefon numarası, T.C. kimlik numarası, geçmiş tarih kontrolü, müşteri kaydı ve aktif paket kontrolü yapılmaktadır.

Veritabanı güvenliği açısından parametreli sorgular kullanılmıştır. Kullanıcıdan gelen değerler SQL sorgularının içine doğrudan metin olarak eklenmemiş, .input yöntemiyle parametre olarak verilmiştir. Bu yaklaşım SQL enjeksiyon riskini azaltmaktadır. Özellikle kullanıcı adı, müşteri arama, id değerleri, tarih bilgileri, fiyat ve tutar alanları gibi dışarıdan gelen verilerin kontrollü şekilde sorguya bağlanması sistem güvenliği açısından önemlidir.

Sistemde veri doğrulama kontrolleri de güvenlik ve veri bütünlüğü açısından önemli rol oynamaktadır. sayiCevir fonksiyonu ile id değerlerinin gerçekten pozitif tam sayı olup olmadığı kontrol edilmektedir. Geçersiz id ile gelen istekler işleme alınmamaktadır. Tarih kontrol fonksiyonları ile geçersiz tarih değerleri engellenmektedir. Online randevu tarafında geçmiş tarih seçimi engellenmektedir. Telefon numarasının 11 haneli ve 0 ile başlaması istenmektedir. Bu kontroller, veritabanına hatalı veya anlamsız veri yazılmasını önlemeye yardımcı olur.

Randevu güvenliği yalnızca kullanıcı girişiyle sınırlı değildir. Sistem iş kuralı güvenliği de uygulamaktadır. Aynı personelin aynı zaman aralığında iki farklı randevuya atanması engellenmektedir. Aynı odanın aynı zaman aralığında iki farklı randevuda kullanılması engellenmektedir. Aynı müşteri için aynı hizmette çakışan randevu oluşturulması kontrol edilmektedir. Online randevu tarafında müşterinin ilgili hizmet için aktif paketi yoksa randevu talebi reddedilmektedir. Bu kontroller, sistemin işletme kurallarına uygun çalışmasını sağlar.

Transaction kullanımı da veri güvenliği açısından önemlidir. Paket satışı veya stok alımı gibi birden fazla tabloyu etkileyen işlemlerde transaction başlatılmakta, tüm adımlar başarılı olursa commit yapılmakta, hata oluşursa rollback ile işlem geri alınmaktadır. Böylece örneğin paket kaydı

oluşup kasa hareketi oluşmaması veya ürün stoğu eklenip gider kaydının eksik kalması gibi tutarsızlıkların önüne geçilmektedir.

Sistem hata yönetimi açısından da temel önlemler içermektedir. Backend tarafında yakalanmayan hatalar için uncaughtException ve unhandledRejection kontrolleri eklenmiştir. Bu sayede kritik hatalar terminalde görülebilmekte, özellikle veritabanı bağlantı problemleri veya beklenmeyen async hatalar tespit edilebilmektedir.

Mevcut güvenlik yapısı proje düzeyi için yeterli bir temel sunsa da, gerçek bir canlı kullanım senaryosunda geliştirilebilecek alanlar da bulunmaktadır. Token bilgileri şu anda bellek üzerinde tutulduğu için sunucu yeniden başladığında oturumlar sıfırlanır. Gelecekte JWT tabanlı oturum yapısı veya veritabanında saklanan oturum yönetimi kullanılabilir. Ayrıca HTTPS kullanımı, daha detaylı rol ve izin matrisi, işlem logları, kullanıcı hareket kayıtları, şifre sıfırlama mekanizması, hesap kilitleme ve IP bazlı güvenlik kontrolleri eklenebilir.

Şekil 1: Şifre Hashleme ve İlk Admin oluşturma Kodları

```
// Sistemde hiç admin yoksa ilk yönetici hesabı bu route ile oluşturulur. Daha önce admin varsa tekrar oluşturulmasına izin verilmez.
app.post("/api/ilk-admin", async (req, res) => {
  try {
    const { ad, soyad, kullanici_adi, sifre } = req.body;

    if (!ad || !soyad || !kullanici_adi || !sifre) {
      return res.status(400).json({
        basarili: false,
        mesaj: "ad, soyad, kullanici_adi ve sifre zorunludur",
      });
    }

    // İlk admin eklenmeden önce veritabanında mevcut admin olup olmadığı kontrol edilir.
    const adminKontrol = await havuz
      .request()
      .query("SELECT TOP 1 id FROM dbo.Kullanicilar WHERE rol = 'Admin'");

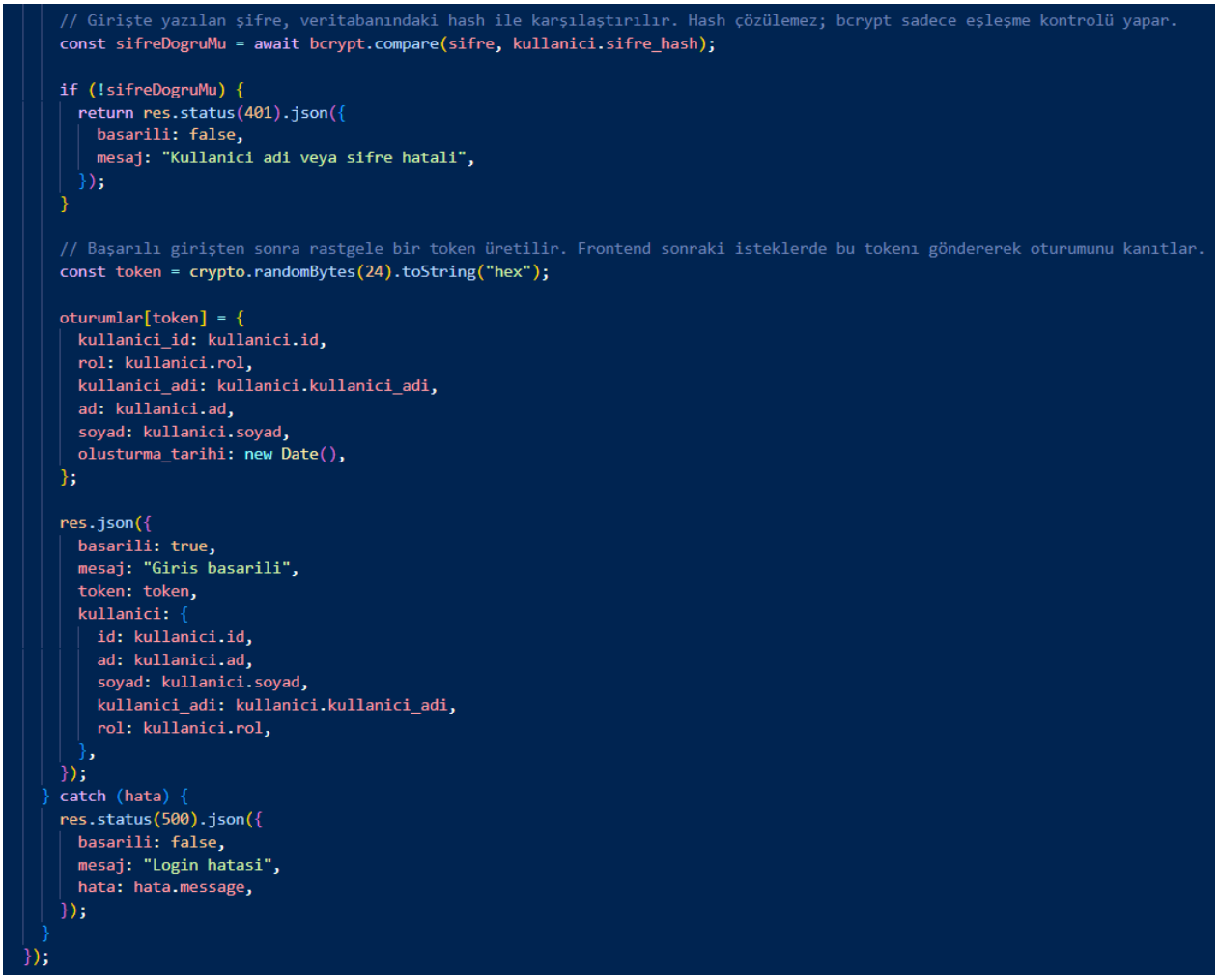
    if (adminKontrol.recordset.length > 0) {
      return res.status(400).json({
        basarili: false,
        mesaj: "Sistemde zaten admin var",
      });
    }
```

```
    const sifreHash = await bcrypt.hash(sifre, 10);

    const sonuc = await havuz
      .request()
      .input("ad", sql.NVarChar(100), ad)
      .input("soyad", sql.NVarChar(100), soyad)
      .input("kullanici_adi", sql.NVarChar(100), kullanici_adi)
      .input("sifre_hash", sql.NVarChar(400), sifreHash)
      .query(`
        INSERT INTO dbo.Kullanicilar
        (ad, soyad, kullanici_adi, sifre_hash, rol, durum)
        OUTPUT INSERTED.id, INSERTED.ad, INSERTED.soyad, INSERTED.kullanici_adi, INSERTED.rol
        VALUES
        (@ad, @soyad, @kullanici_adi, @sifre_hash, 'Admin', 1)
      `);

    res.status(201).json({
      basarili: true,
      mesaj: "Ilk admin olusturuldu",
      veri: sonuc.recordset[0],
    });
  } catch (hata) {
    res.status(500).json({
      basarili: false,
      mesaj: "Ilk admin olusturulamadi",
      hata: hata.message,
    });
  }
});
```

Şekil 2: Token Üretmeyi Sağlayan Kod Bloğu



```
    // Girişte yazılan şifre, veritabanındaki hash ile karşılaştırılır. Hash çözülemez; bcrypt sadece eşleşme kontrolü yapar.
    const sifreDogruMu = await bcrypt.compare(sifre, kullanici.sifre_hash);

    if (!sifreDogruMu) {
      return res.status(401).json({
        basarili: false,
        mesaj: "Kullanici adi veya sifre hatali",
      });
    }

    // Başarılı girişten sonra rastgele bir token üretilir. Frontend sonraki isteklerde bu tokenı göndererek oturumunu kanıtlar.
    const token = crypto.randomBytes(24).toString("hex");

    oturumlar[token] = {
      kullanici_id: kullanici.id,
      rol: kullanici.rol,
      kullanici_adi: kullanici.kullanici_adi,
      ad: kullanici.ad,
      soyad: kullanici.soyad,
      olusturma_tarihi: new Date(),
    };

    res.json({
      basarili: true,
      mesaj: "Giris basarili",
      token: token,
      kullanici: {
        id: kullanici.id,
        ad: kullanici.ad,
        soyad: kullanici.soyad,
        kullanici_adi: kullanici.kullanici_adi,
        rol: kullanici.rol,
      },
    });
  } catch (hata) {
    res.status(500).json({
      basarili: false,
      mesaj: "Login hatasi",
      hata: hata.message,
    });
  }
});
```

Şekil 3: girisGerekli Ve sadeceAdmin Fonksiyonları

```
function girisGerekli(req, res, next) {
  const auth = req.headers.authorization;

  if (!auth || !auth.startsWith("Bearer ")) {
    return res.status(401).json({
      basarili: false,
      mesaj: "Token yok. Once giris yapmalisin.",
    });
  }
```

```
function sadeceAdmin(req, res, next) {
  if (req.kullanici.rol !== "Admin") {
    return res.status(403).json({
      basarili: false,
      mesaj: "Bu islem icin Admin yetkisi gerekir.",
    });
  }

  next();
}
```

6. SONUÇ VE GELECEK GELİŞTİRMELER

ShineBeauty projesi, güzellik merkezi işletmelerinin operasyonel ve finansal süreçlerini dijital ortamda yönetebilmesi için geliştirilmiş kapsamlı bir yönetim bilgi sistemi olarak tamamlanmıştır. Proje kapsamında müşteri kayıtları, personel yönetimi, hizmet ve hizmet seçeneği yönetimi, oda yönetimi, randevu planlama, online randevu talebi, paket seans takibi, ürün ve stok yönetimi, ürün satışı, gider takibi, kasa hareketleri, dashboard ve raporlama gibi temel iş süreçleri tek sistem altında toplanmıştır.

Projenin en güçlü yönlerinden biri, modüllerin birbirinden bağımsız görünmesine rağmen veritabanı ve API yapısı üzerinden birlikte çalışmasıdır. Randevu modülü müşteri, personel, hizmet, hizmet seçeneği ve oda tablolarıyla ilişkilidir. Ürün satışı modülü hem stok miktarını değiştirir hem de finansal gelir oluşturur. Paket satışı hem müşteriye seans hakkı tanımlar hem de kasaya gelir olarak yansır. Randevu tamamlandığında ilgili paketten seans düşülebilir ve bu

kullanım ayrıca kayıt altına alınabilir. Bu yapı, sistemin yalnızca görsel bir panel olmadığını, gerçek işletme akışlarını birbirine bağlayan entegre bir yazılım olduğunu göstermektedir.

Sistem, manuel takipte karşılaşılabilecek birçok hatayı azaltacak kontroller içermektedir. Randevu oluştururken oda ve personel çakışması kontrol edilmektedir. Online randevu tarafında geçmiş tarih seçimi engellenmektedir. Müşterinin aktif paketi yoksa web üzerinden randevu oluşturmasına izin verilmemektedir. Ürün satışı yapıldığında stok otomatik düşmektedir. Paket satışı ve ürün stoğu gibi çoklu tablo işlemlerinde transaction kullanılarak veri tutarlılığı korunmaktadır. Bu özellikler, projenin işletme süreçlerini güvenli ve kontrollü hale getirme amacına hizmet etmektedir.

Güvenlik tarafında kullanıcı şifrelerinin hashlenerek saklanması, token tabanlı oturum kontrolü, giriş gerektiren API adresleri, admin rolüne özel işlemler ve parametreli SQL sorguları sistemin temel güvenlik yapısını oluşturmaktadır. Bu yapı, proje düzeyinde rol bazlı erişim kontrolünün sağlandığını ve kullanıcıların yetkilerine göre işlem yapabildiğini göstermektedir.

Projenin frontend tarafında yönetim paneli ve public müşteri tarafı ayrı şekilde düşünülmüştür. Yönetim paneli üzerinden işletme çalışanları müşteri, randevu, hizmet, oda, ürün, paket ve finans işlemlerini yönetebilirken; public tarafta ziyaretçiler hizmetleri görebilmekte, ürünleri inceleyebilmekte ve online randevu talebi oluşturabilmektedir. Bu ayrım, projenin hem işletme içi kullanım hem de müşteri etkileşimi açısından daha kapsamlı olmasını sağlamıştır.

Gelecekte proje daha da geliştirilebilir. İlk olarak oturum yönetimi daha profesyonel hale getirilebilir. Bellekte tutulan token sistemi yerine JWT veya veritabanı tabanlı oturum yönetimi kullanılabilir. Böylece oturumlar daha kontrollü ve ölçeklenebilir şekilde yönetilebilir. Ayrıca kullanıcının yaptığı işlemleri kayıt altına alan detaylı log sistemi eklenebilir. Bu log sistemi sayesinde hangi kullanıcının ne zaman hangi kaydı eklediği, güncellediği veya sildiği izlenebilir.

Randevu tarafında SMS, e-posta veya WhatsApp bildirim sistemi eklenebilir. Müşteriye randevu onayı, randevu hatırlatması, iptal bildirimi veya paket seansı bilgilendirmesi otomatik gönderilebilir. Bu geliştirme, müşteri memnuniyetini artıracağı gibi randevuya gelmeme oranını da azaltabilir. Ayrıca Google Takvim senkronizasyonu eklenerek işletmenin günlük randevu planı harici takvim uygulamalarıyla uyumlu hale getirilebilir.

Finans tarafında daha gelişmiş raporlar hazırlanabilir. Günlük, haftalık, aylık ve yıllık ciro raporları grafiklerle gösterilebilir. En çok gelir getiren hizmetler, en çok satış yapan personeller, en çok tercih edilen hizmet seçenekleri, en çok paket satın alan müşteriler ve stok maliyetleri gibi analizler eklenebilir. Ayrıca Excel ve PDF dışa aktarma özellikleri geliştirilerek raporların dışarıya alınması sağlanabilir.

Stok yönetimi tarafında minimum stok seviyesi, tedarikçi bilgisi, ürün kategori yönetimi ve stok giriş-çıkış geçmişi gibi detaylar eklenebilir. Stok belirli bir limitin altına düştüğünde sistem otomatik uyarı verebilir. Ürün alışları, ürün satışları ve stok fireleri ayrı ayrı takip edilebilir.

Paket yönetimi tarafında müşteriye özel paket geçmişi, kalan seans hatırlatması, paket bitiş tarihi, paket dondurma veya iptal etme gibi özellikler eklenebilir. Böylece seans bazlı hizmet veren güzellik merkezleri için daha detaylı bir takip sistemi oluşturulabilir.

Fatura ve ödeme tarafında mevcut PDF üretim yapısı daha da geliştirilerek e-fatura veya e-arşiv simülasyonu yapılabilir. Sanal POS simülasyonu veya gerçek ödeme entegrasyonu eklenerek paket ve ürün satışlarının online ödeme ile tamamlanması sağlanabilir. Bu özellik, projenin ticari kullanım kapasitesini artıracaktır.

Sonuç olarak ShineBeauty projesi, güzellik merkezi işletmelerinin temel ihtiyaçlarını karşılayan, modüler, ilişkisel veritabanı yapısına sahip, rol bazlı erişim kullanan ve işletme süreçlerini dijitalleştiren başarılı bir yönetim bilgi sistemi örneğidir. Proje mevcut haliyle müşteri,

randevu, personel, hizmet, oda, paket, ürün, stok ve finans işlemlerini çalışır şekilde bir araya getirmektedir. Gelecekte bildirim, gelişmiş raporlama, daha güçlü güvenlik, ödeme entegrasyonu, takvim senkronizasyonu ve gelişmiş fatura özellikleri eklenerek daha profesyonel ve gerçek işletme kullanımına daha yakın bir sisteme dönüştürülebilir.

KAYNAKÇA

Govett, D. (t.y.). PDFKit documentation. PDFKit. pdfkit.org/

kelektiv. (t.y.). bcrypt. npm. www.npmjs.com/package/bcrypt

Microsoft. (t.y.). SQL Server technical documentation. Microsoft Learn. learn.microsoft.com/tr-tr/sql-server/

OpenJS Foundation. (t.y.). Express Node.js web application framework. Express.js. expressjs.com/

OpenJS Foundation. (t.y.). Node.js documentation. Node.js. nodejs.org/en/docs/

tediousjs. (t.y.). mssql. npm. www.npmjs.com/package/mssql

TimelordUK. (t.y.). msnodesqlv8. npm. www.npmjs.com/package/msnodesqlv8